

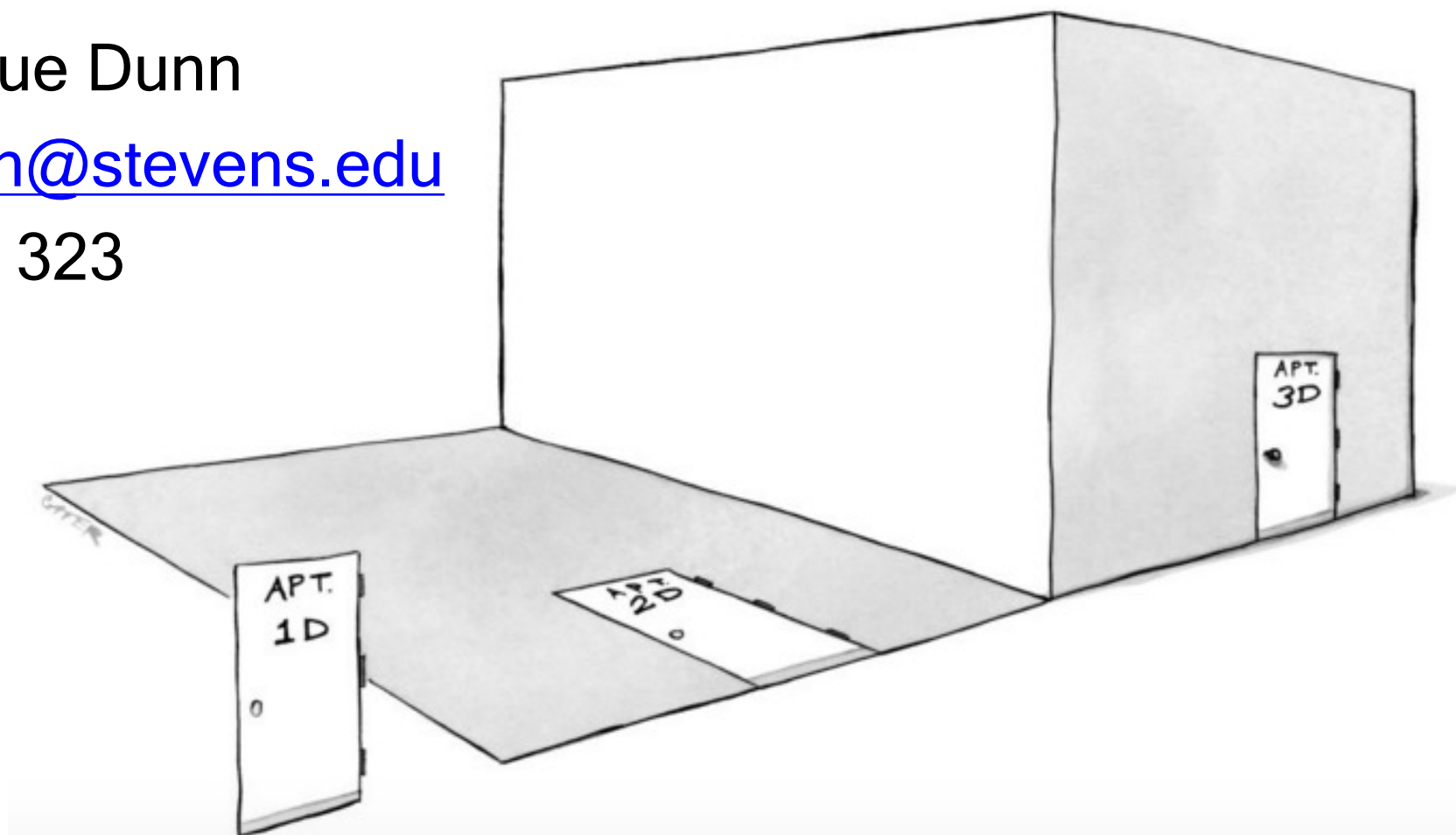
CS 532: 3D Computer Vision

Lecture 11

Enrique Dunn

edunn@stevens.edu

GSW 323



Lecture Outline

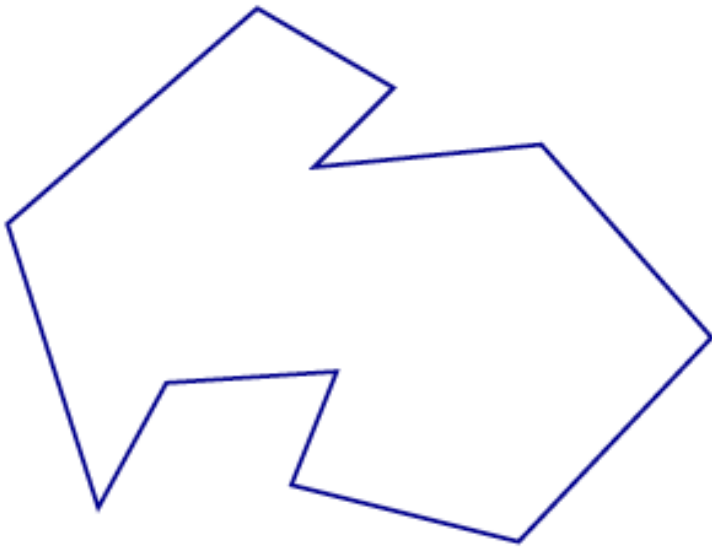
- Triangulating a Polygon
- Voronoi diagrams
- Delaunay triangulations
 - David M. Mount, CMSC 754: Computational Geometry lecture notes, Department of Computer Science, University of Maryland, Spring 2012
 - Lectures 11, 12 and 13
 - Slides by:
 - M. van Kreveld (Utrecht University)

Triangulating a Polygon (intro)

Slides by M. van Kreveld

Simple Polygon

- A polygonal curve is a finite chain of line segments.
- Line segments called edges, their endpoints called vertices.
- A simple polygon is a closed polygonal curve without self-intersection.



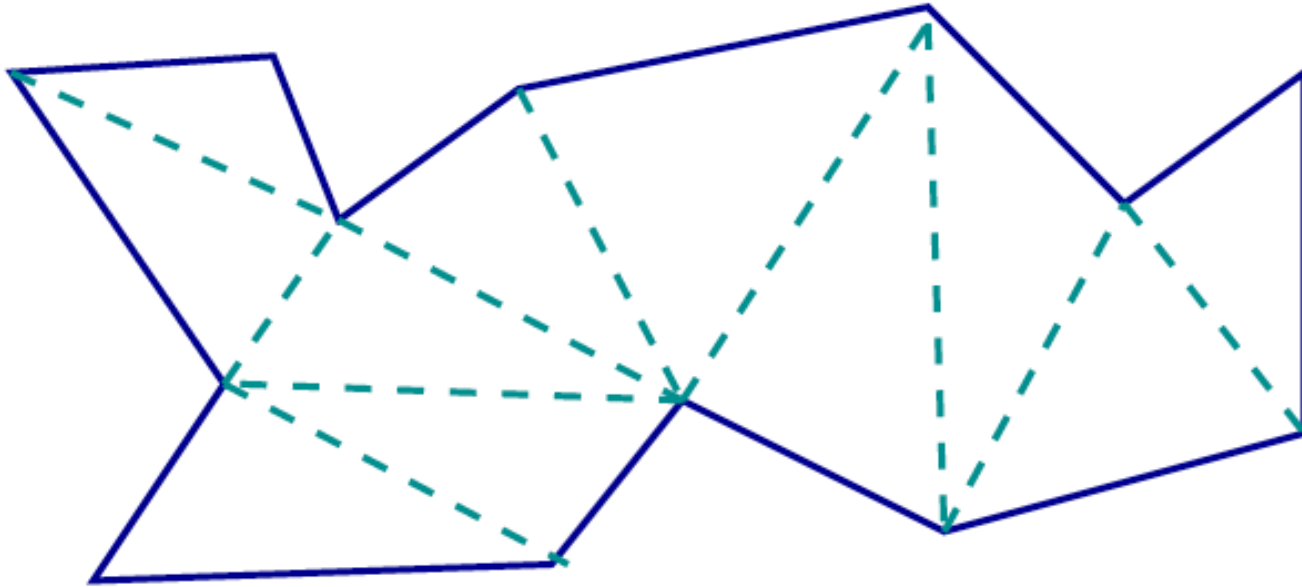
Simple Polygon



Non-Simple Polygons

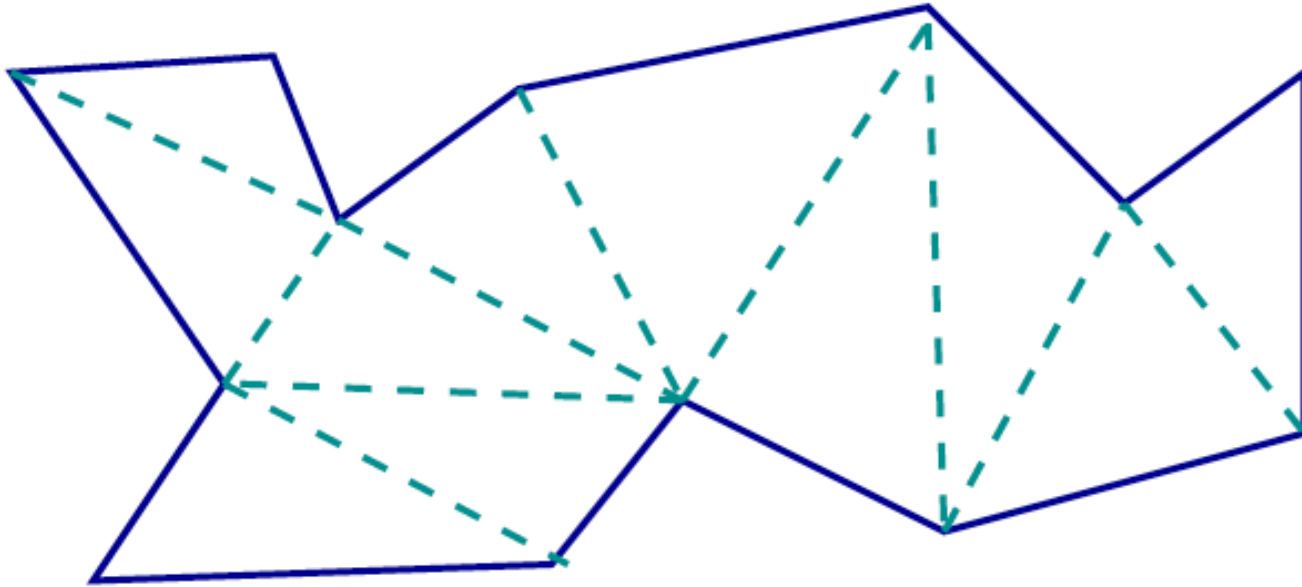
Triangulation

Partition polygon P into non-overlapping triangles using diagonals only.



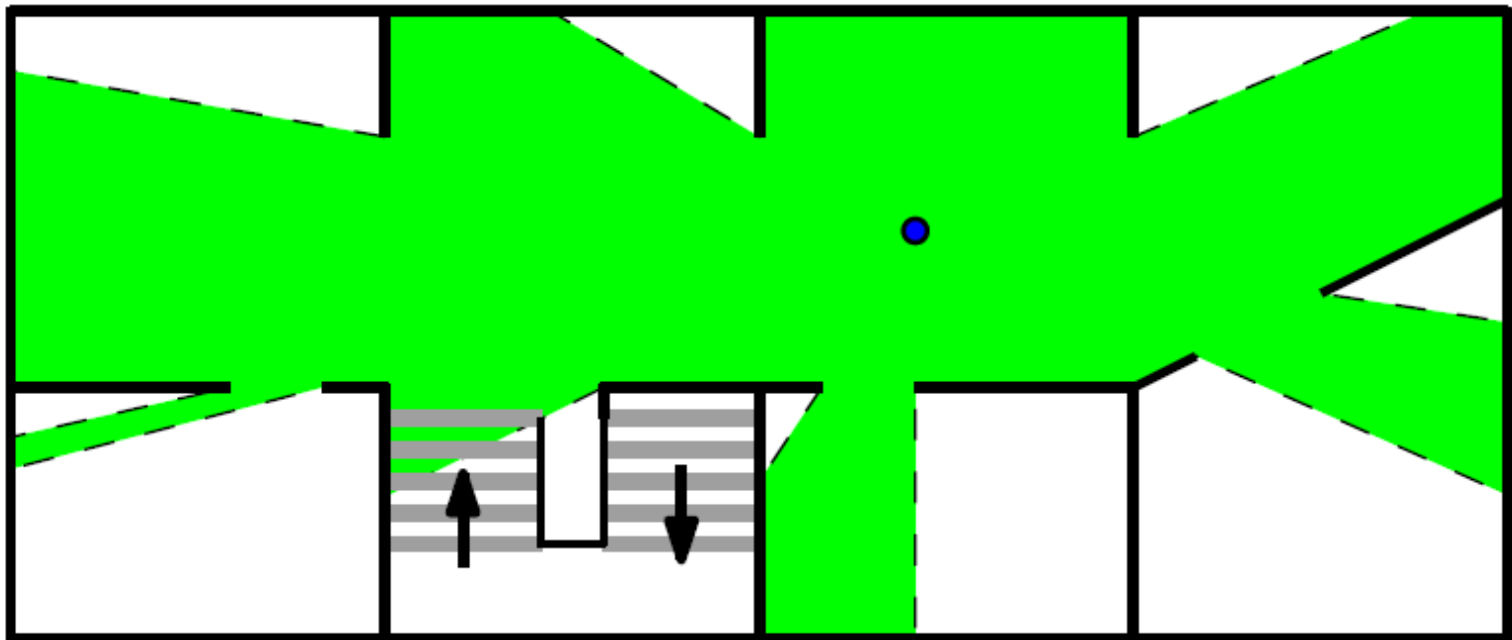
Triangulation

Every simple polygon admits a triangulation.
Every triangulation of an n -gon has exactly $n-2$ triangles.



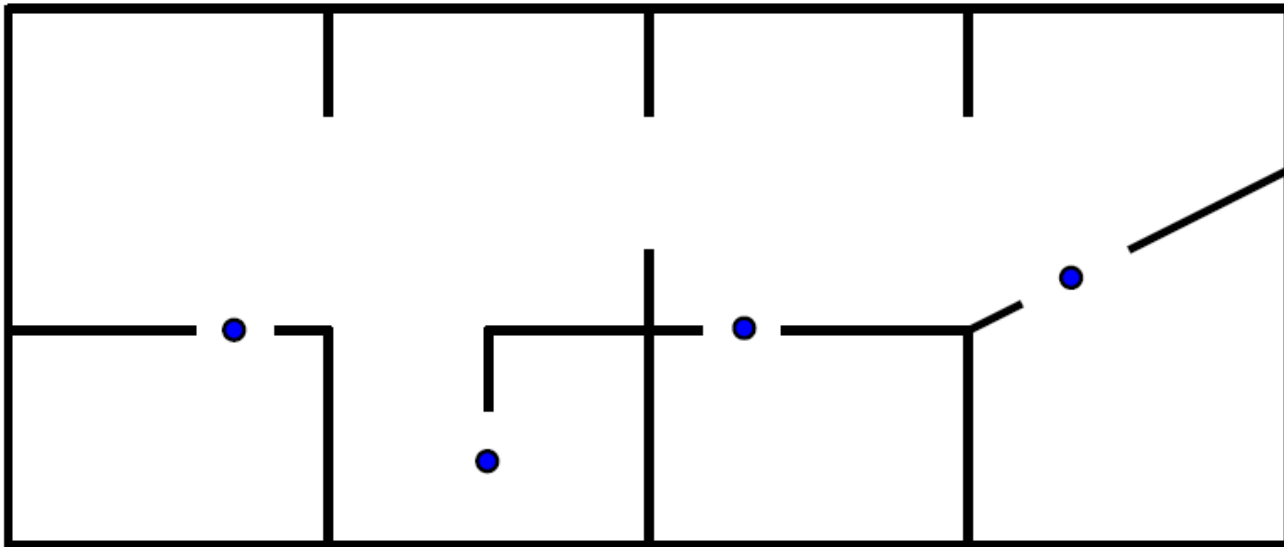
Polygons and Visibility

- Two points in a simple polygon can see each other if their connecting line segment is in the polygon



The Art Gallery Problem

- How many cameras are needed to guard a given art gallery so that every point is seen?

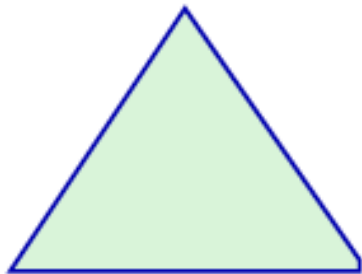


The Art Gallery Problem

- In geometry terminology: How many points are needed in a simple polygon with n vertices so that every point in the polygon is seen?
- The optimization problem is computationally difficult
- Art Gallery Theorem: $\lfloor n/3 \rfloor$ cameras are occasionally necessary but always sufficient

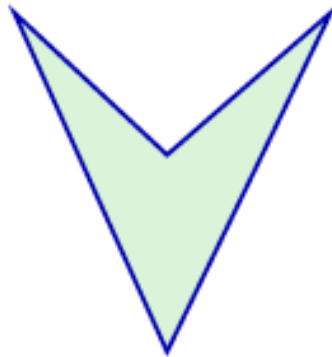
Examples

$n = 3$



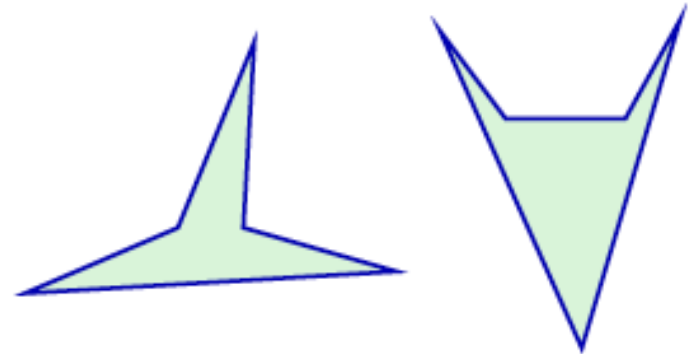
$g(n) = 1$

$n = 4$



$g(n) = 1$

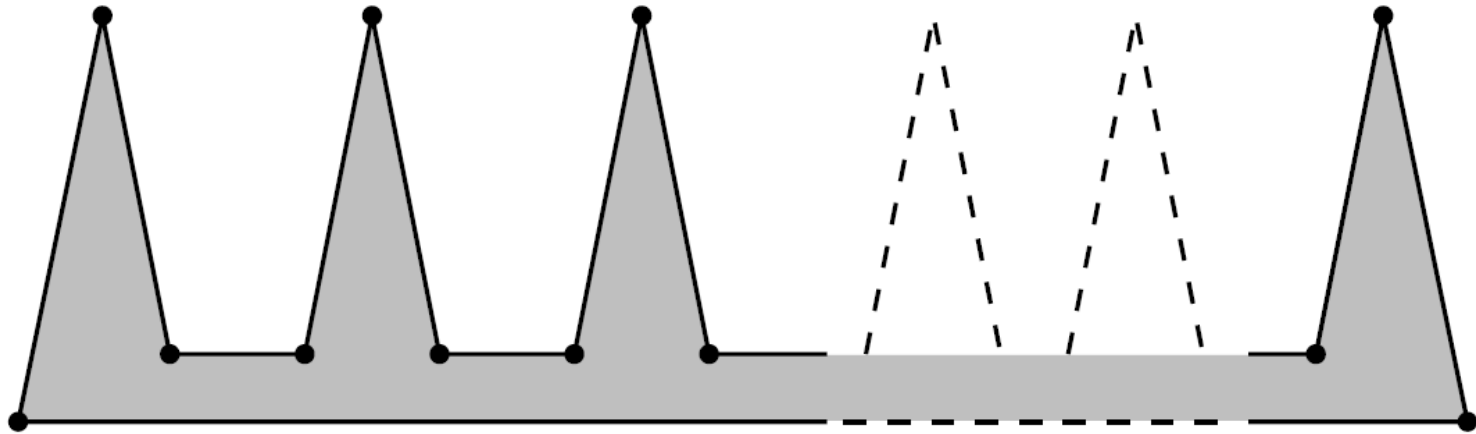
$n = 5$



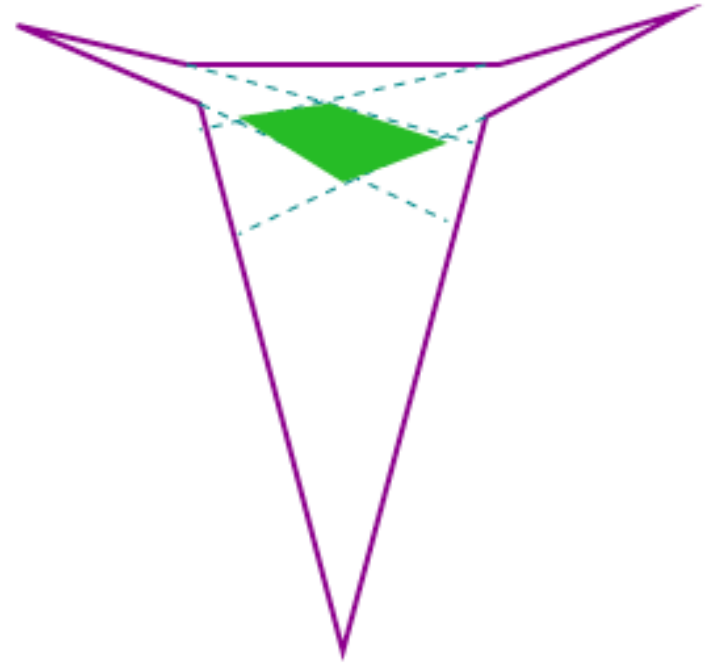
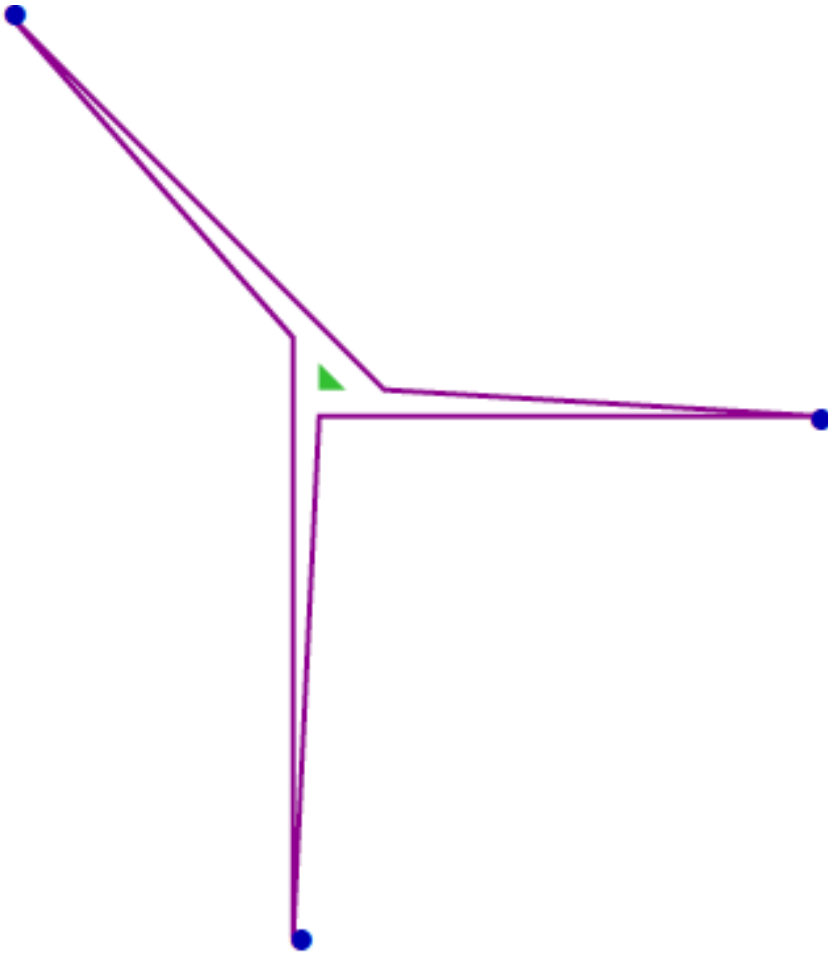
$g(n) = 1$

The Art Gallery Problem

- Art Gallery Theorem: $\lfloor n/3 \rfloor$ cameras are occasionally necessary but always sufficient

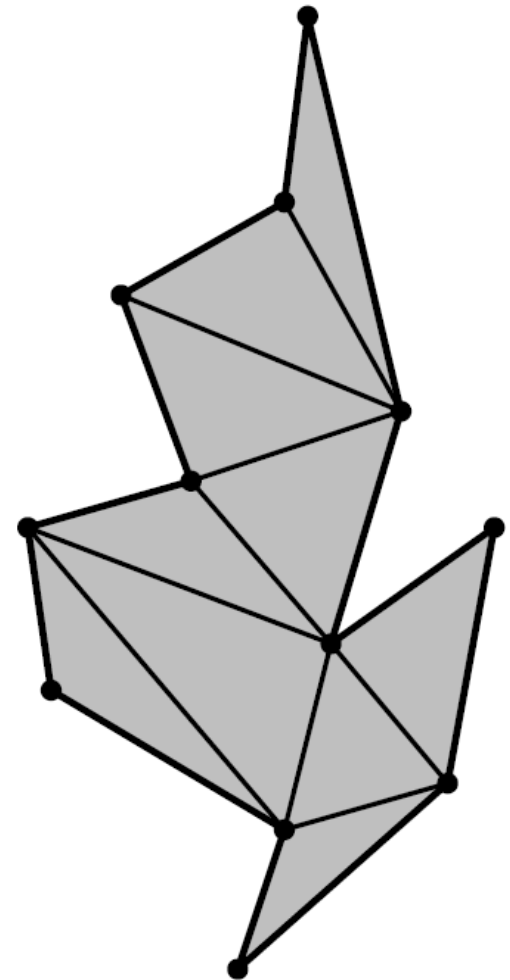


Examples



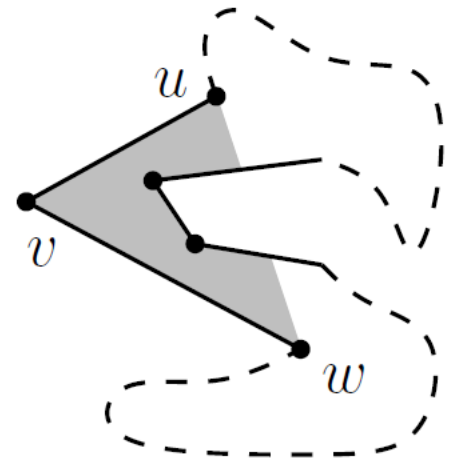
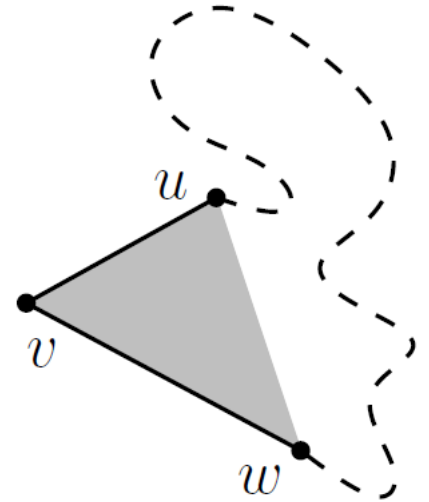
Diagonals

- Why are $\lfloor n/3 \rfloor$ cameras always enough?
- Assume polygon P is triangulated: a decomposition of P into disjoint triangles by a maximal set of non-intersecting diagonals
- Diagonal of P : open line segment that connects two vertices of P and fully lies in the interior of P



A Triangulation Always Exists

- Lemma: A simple polygon with n vertices can always be triangulated, and always with $n-2$ triangles
- Proof: Induction on n . If $n = 3$, it is trivial
- Assume $n > 3$. Consider the leftmost vertex v and its two neighbors u and w .
- Either uw is a diagonal (case 1), or part of the boundary of P is in Δuvw (case 2)
- Case 2: choose the vertex t in Δuvw farthest from the line through u and w , then vt must be a diagonal

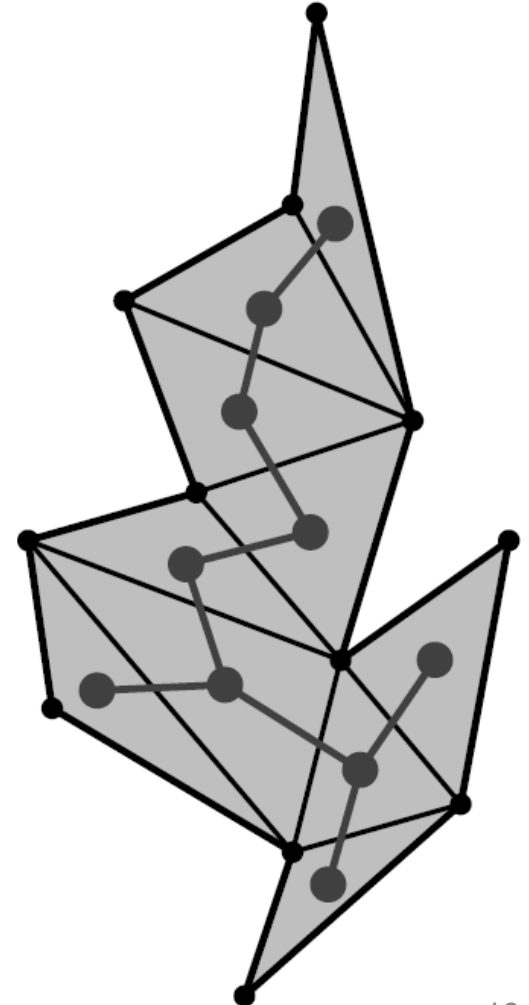


A Triangulation Always Exists

- In case 1, uw cuts the polygon into a triangle and a simple polygon with $n-1$ vertices, and we apply induction
- In case 2, vt cuts the polygon into two simple polygons with m and $n - m + 2$ vertices, $3 \leq m \leq n - 1$, and we also apply induction
- By induction, the two polygons can be triangulated using $m - 2$ and $n - m + 2 - 2 = n - m$ triangles. So the original polygon is triangulated using $m - 2 + n - m = n - 2$ triangles

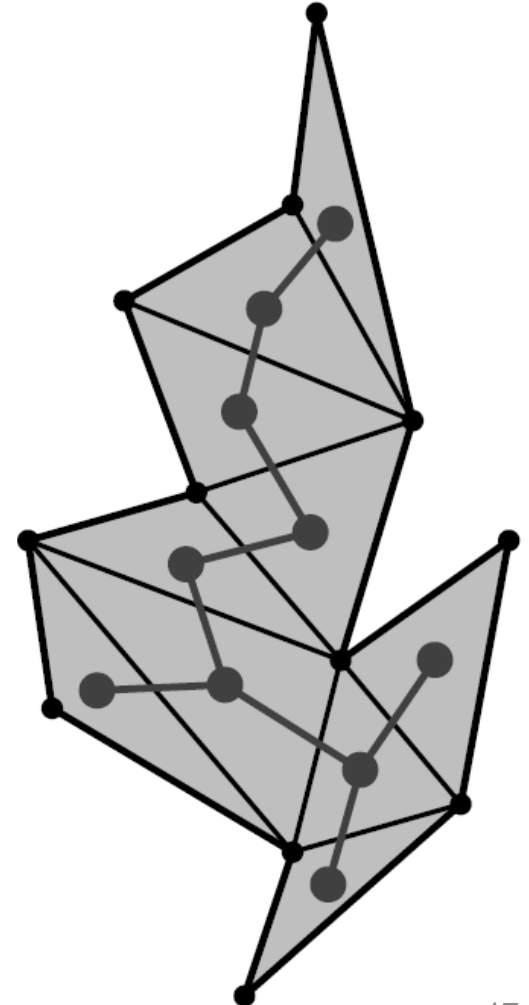
A 3-coloring Always Exists

- Observe: the dual graph of a triangulated simple polygon is a tree
- Dual graph: each face gives a node; two nodes are connected if the faces are adjacent

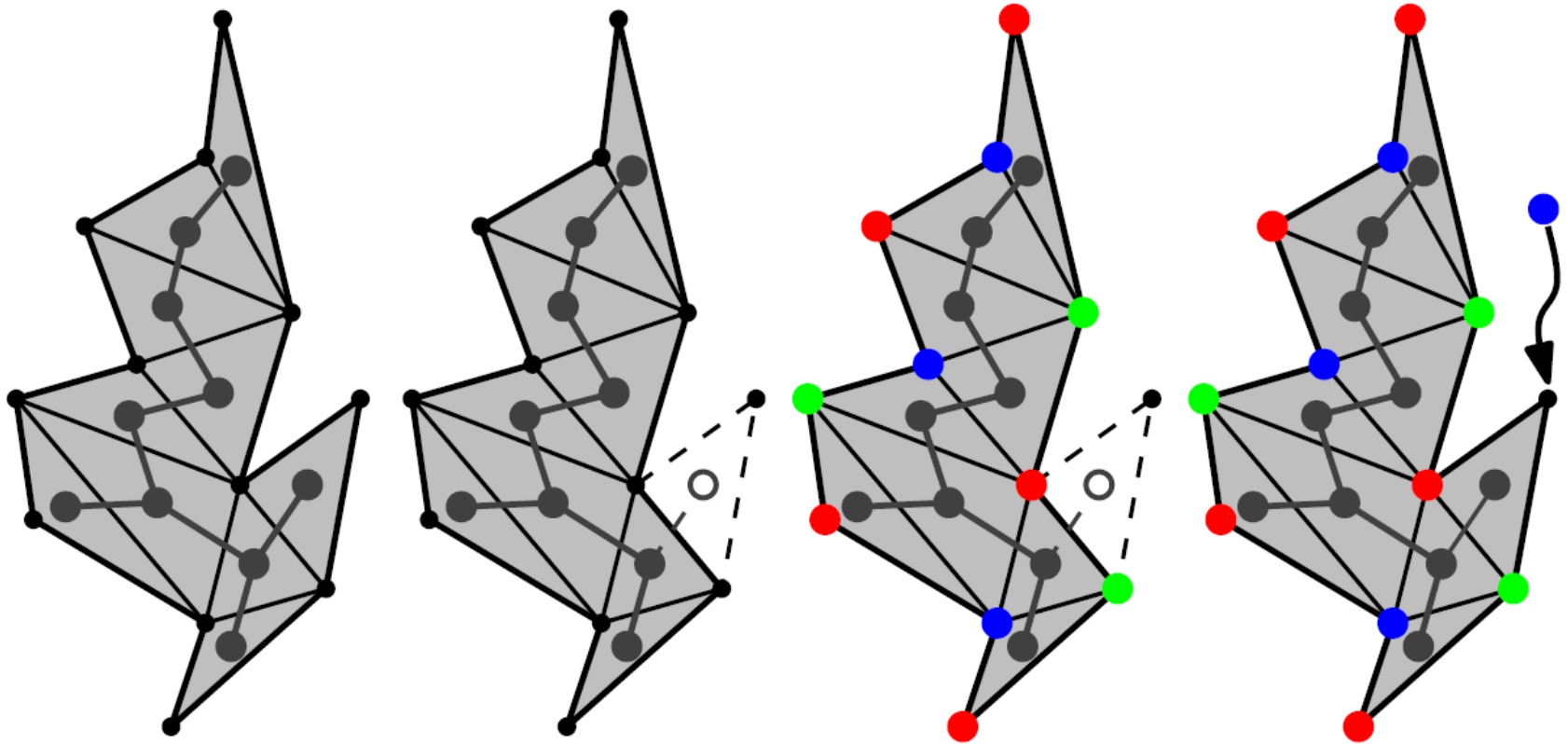


A 3-coloring Always Exists

- Lemma: The vertices of a triangulated simple polygon can always be 3-colored
- Proof: Induction on the number of triangles in the triangulation. Base case: True for a triangle
- Every tree has a leaf. Remove the corresponding triangle from the triangulated polygon, color its vertices, add the triangle back, and let the extra vertex of the neighboring triangle have the color that is not present at its neighbors

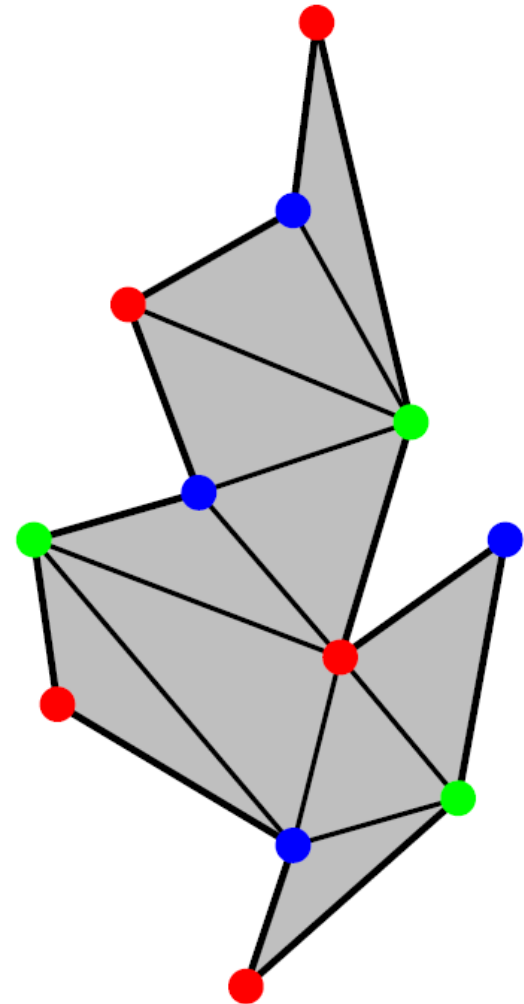


A 3-coloring Always Exists



$\lfloor n/3 \rfloor$ Cameras are Enough

- For a 3-colored, triangulated simple polygon, one of the color classes is used by at most $\lfloor n/3 \rfloor$ colors.
 - Place the cameras at these vertices
- This argument is called the pigeon-hole principle
- Why does the proof fail when the polygon has holes?



Triangulating a Polygon

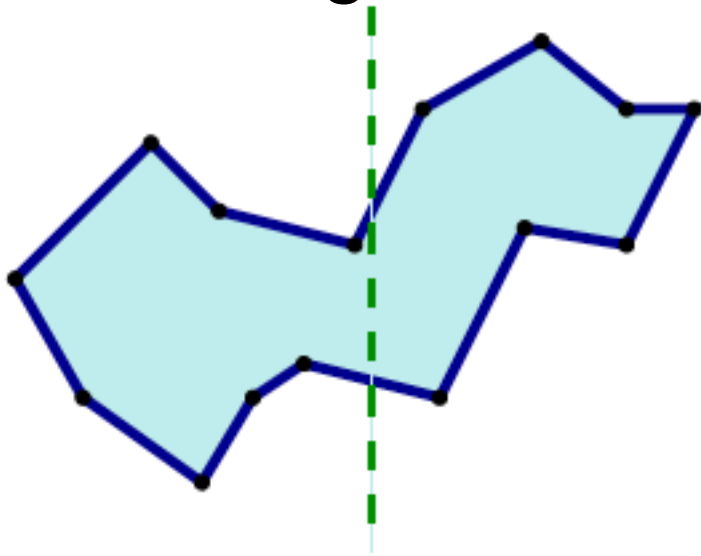
Slides by M. van Kreveld (Utrecht
University)

History

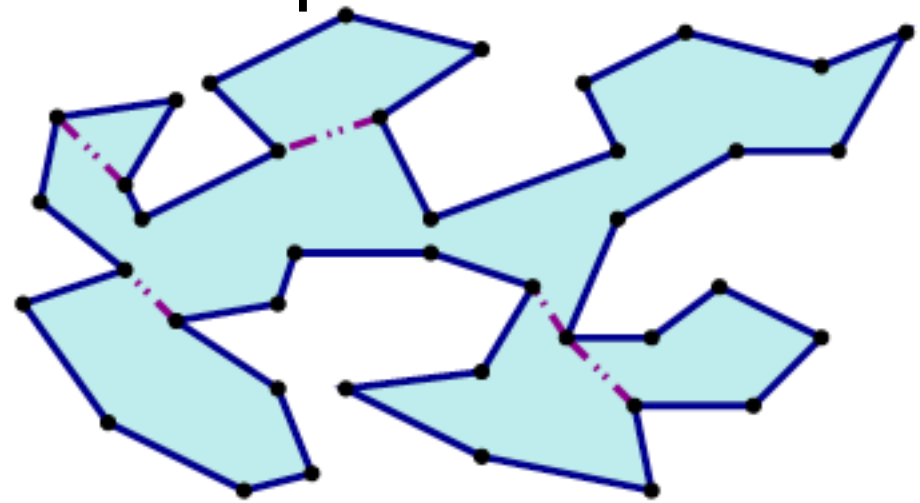
- A really naive algorithm is $O(n^4)$:
 - check all n^2 choices for a diagonal, each in $O(n)$ time. Repeat this $n - 1$ times.
- A better naive algorithm is $O(n^2)$;
 - find an ear in $O(n)$ time; then recurse.
- First non-trivial algorithm: $O(n \log n)$
[GJPT-78]
- Linear complexity algorithms do exist...

Algorithm

- Partition polygon into trapezoids.
- Convert trapezoids into monotone subdivision.
- Triangulate each monotone piece



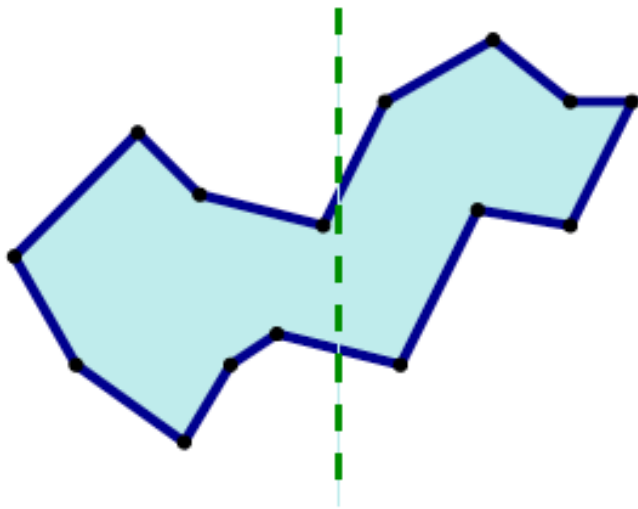
x-monotone polygon



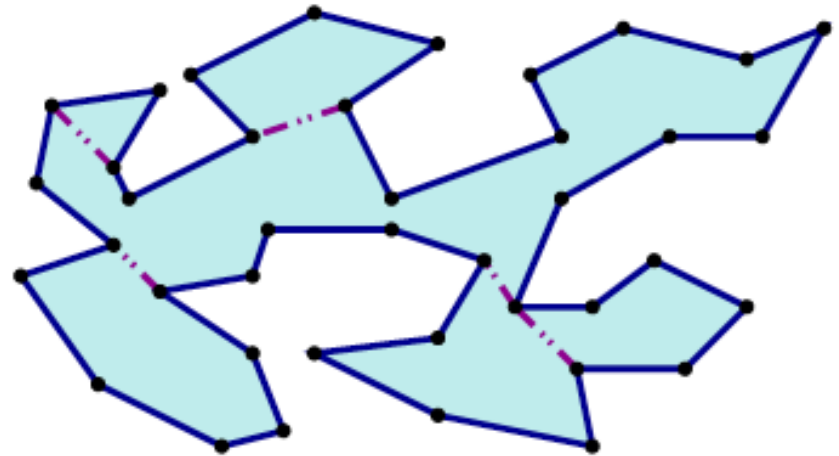
Monotone decomposition

Monotone Polygons

1. A polygonal chain C is monotone w.r.t. line L if any line orthogonal to L intersects C in at most one point.
 - A polygon is monotone w.r.t. L if it can be decomposed into two chains, each monotone w.r.t. L .



x-monotone polygon

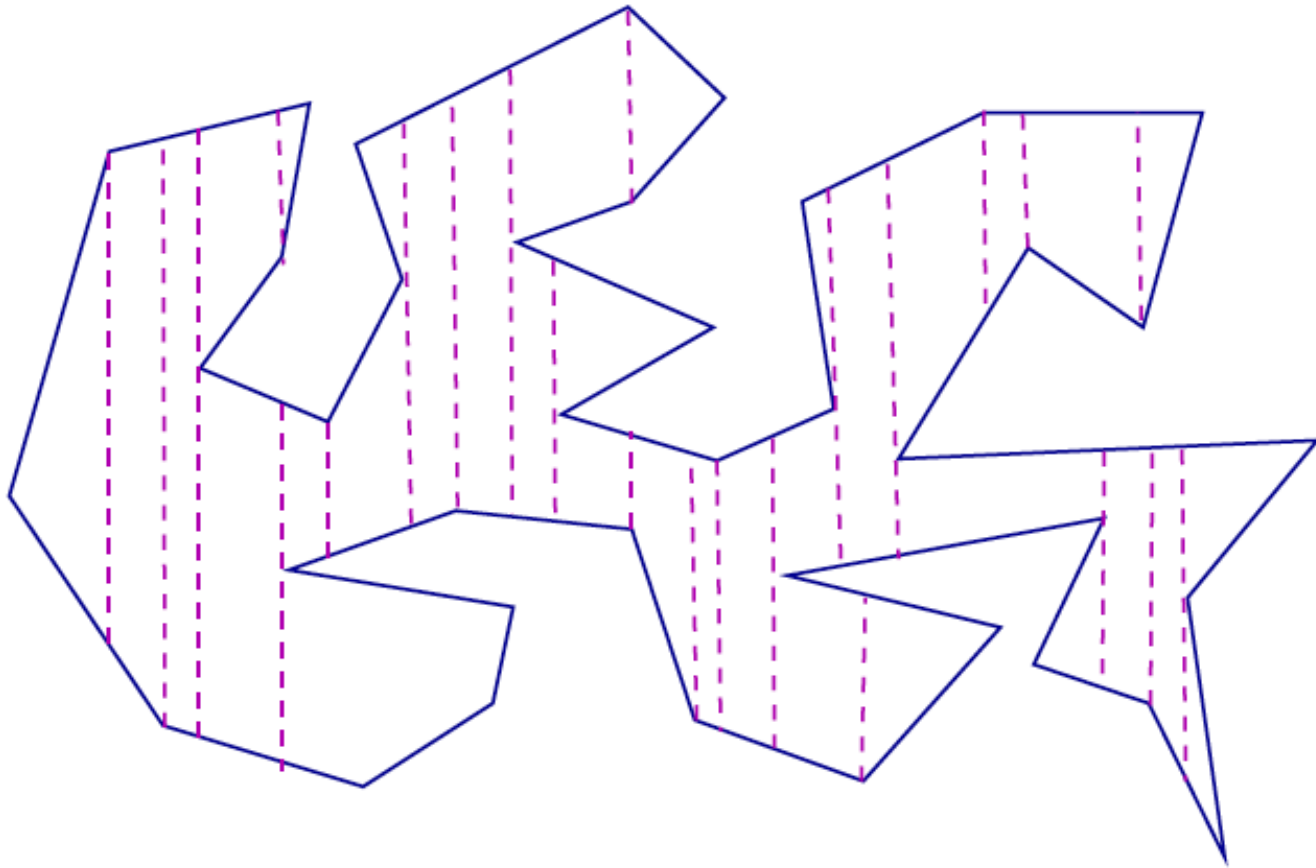


Monotone decomposition

Trapezoidal Decomposition

- At each vertex, extend vertical line until it hits a polygon edge.
- Each face of this decomposition is a trapezoid; which may degenerate into a triangle.
- Time complexity is $O(n \log n)$.

Trapezoidal Decomposition



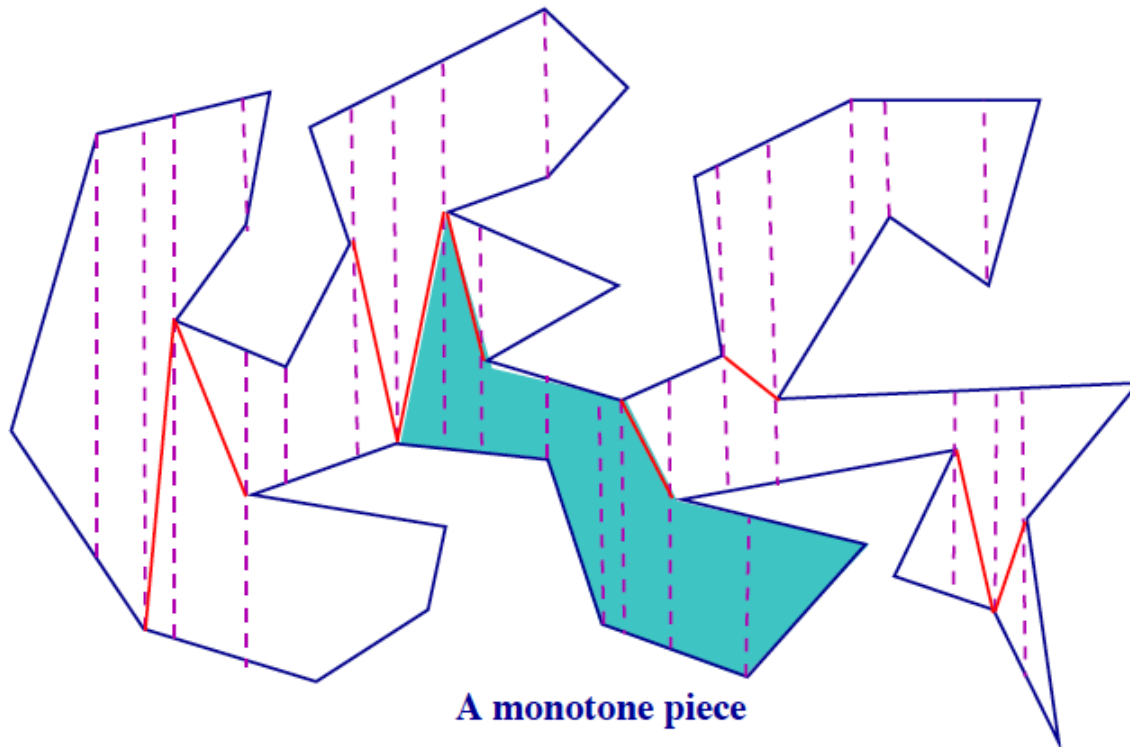
Monotone subdivision

- Call a reflex vertex with both rightward (leftward) edges a split (merge) vertex.
- Non-monotonicity comes from split or merge vertices.
- Add a diagonal to each to remove the non-monotonicity.

▪

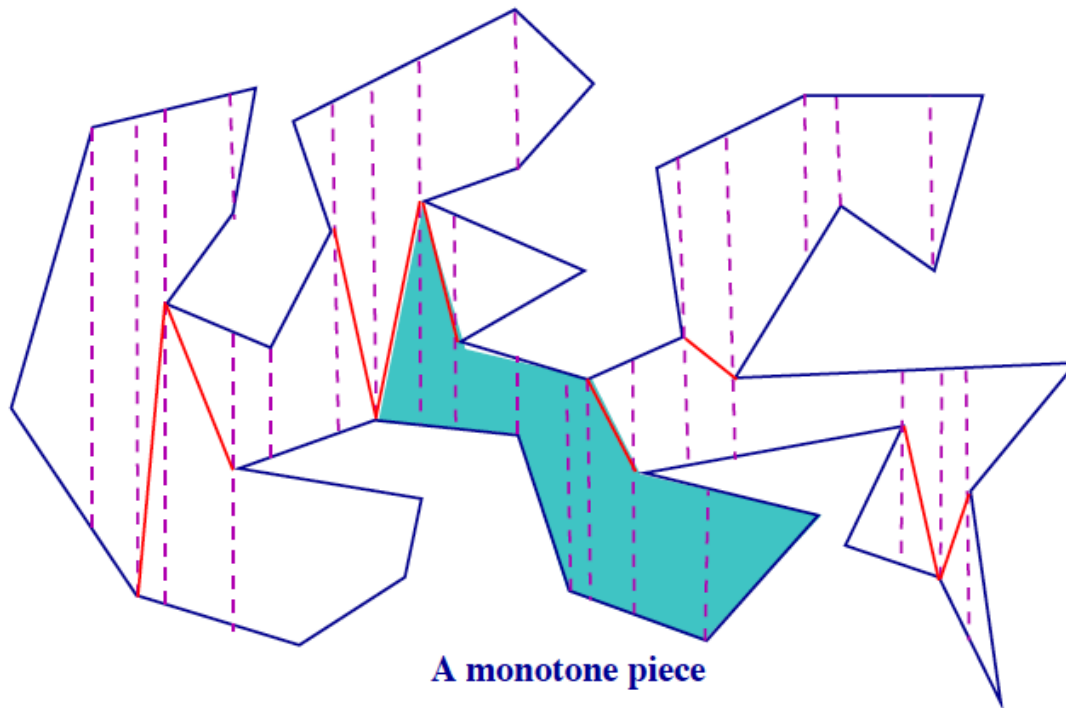
Monotone subdivision

- To each split (merge) vertex, add a diagonal joining it to the polygon vertex of its left (right) trapezoid.



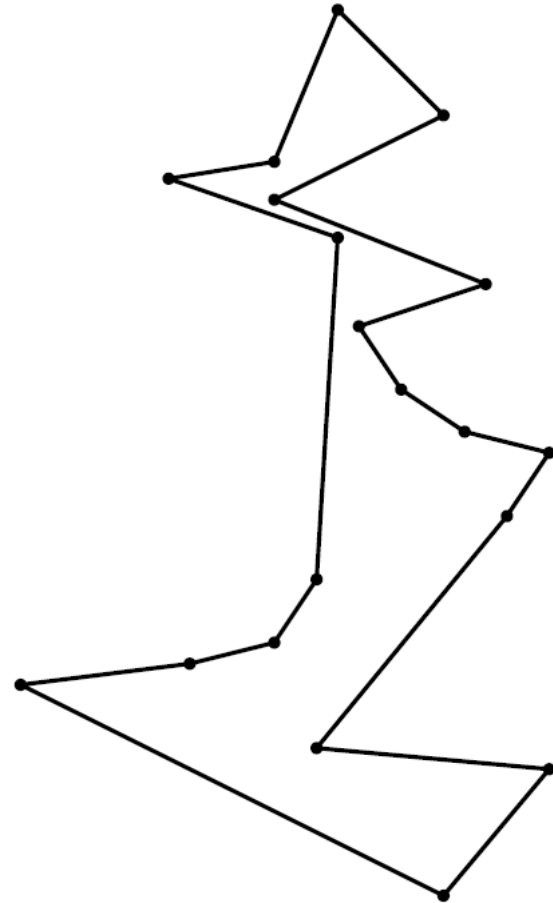
Monotone subdivision

- Remove all trapezoidal edges.
- The polygon boundary plus new split/merge edges form the monotone subdivision.



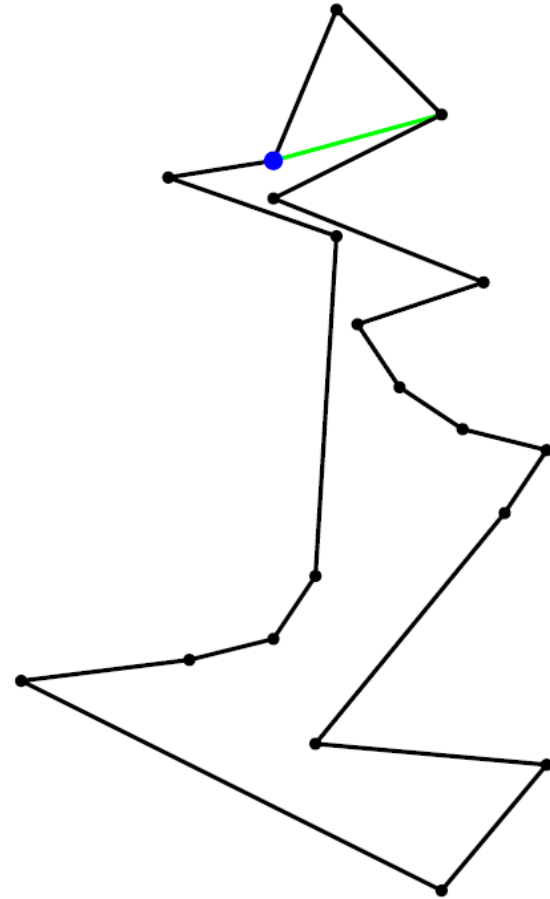
Triangulating a Monotone Polygon

- How to triangulate a y-monotone polygon?



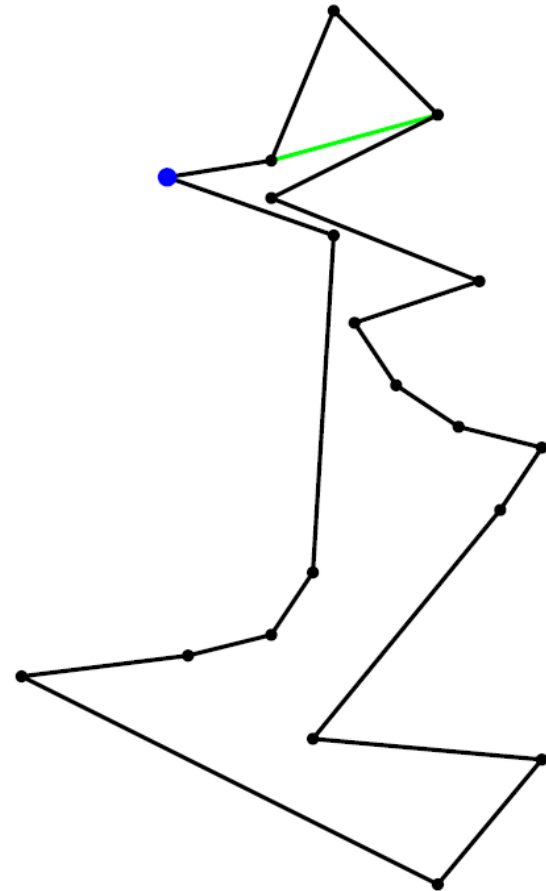
Triangulating a Monotone Polygon

- How to triangulate a y-monotone polygon?



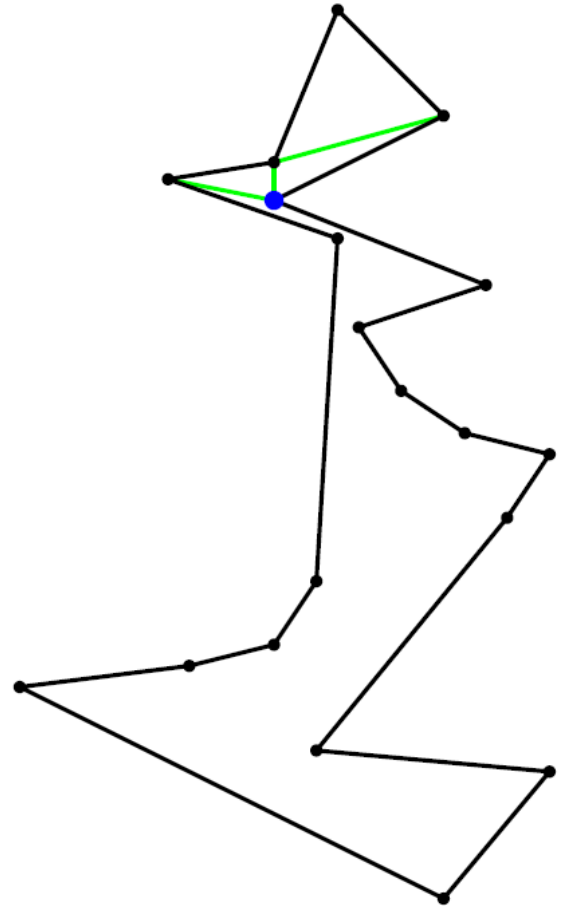
Triangulating a Monotone Polygon

- How to triangulate a y-monotone polygon?



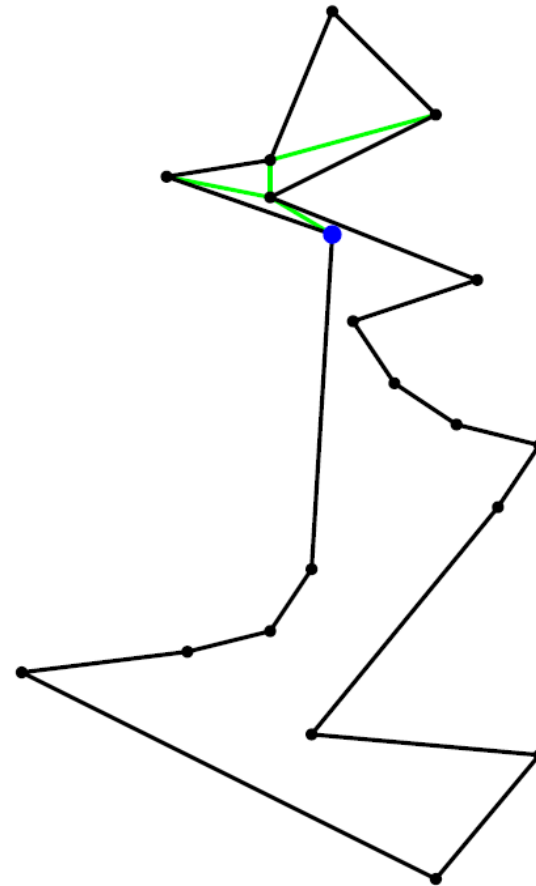
Triangulating a Monotone Polygon

- How to triangulate a y-monotone polygon?



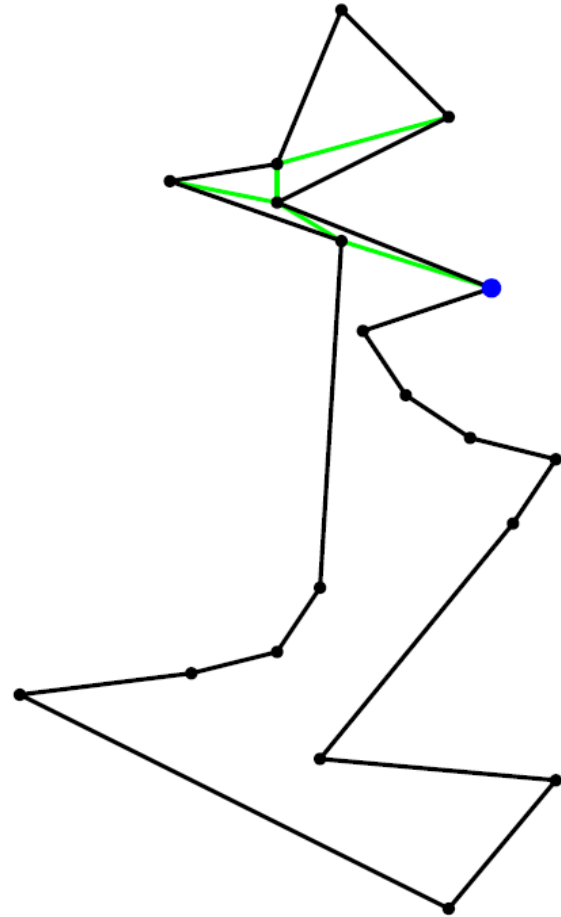
Triangulating a Monotone Polygon

- How to triangulate a y-monotone polygon?



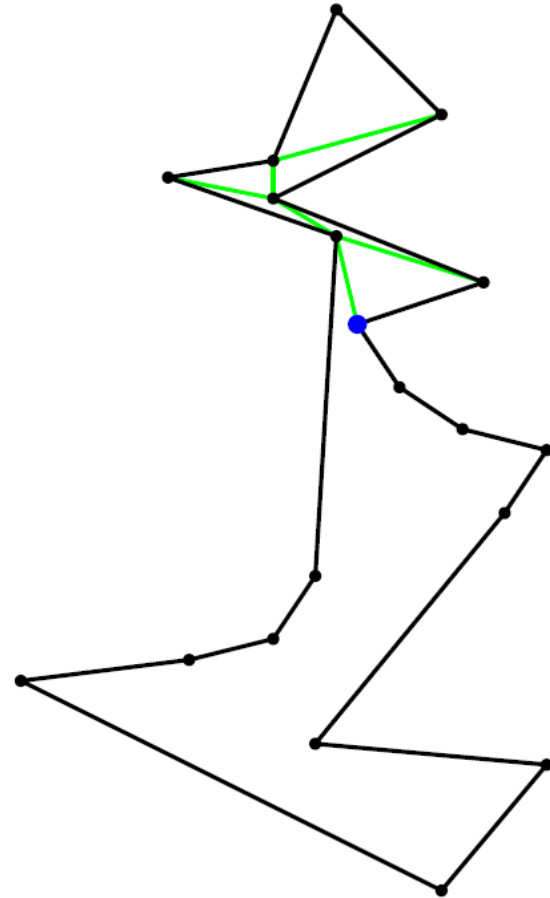
Triangulating a Monotone Polygon

- How to triangulate a y-monotone polygon?



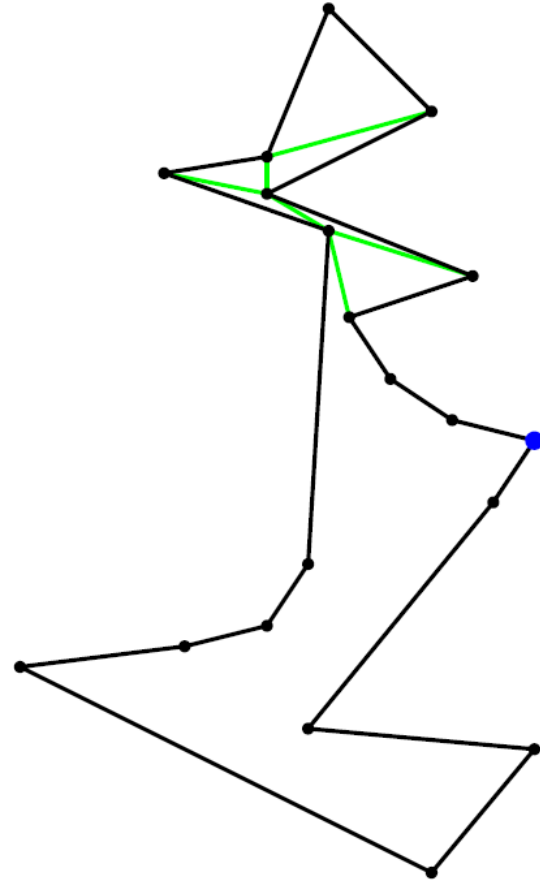
Triangulating a Monotone Polygon

- How to triangulate a y-monotone polygon?



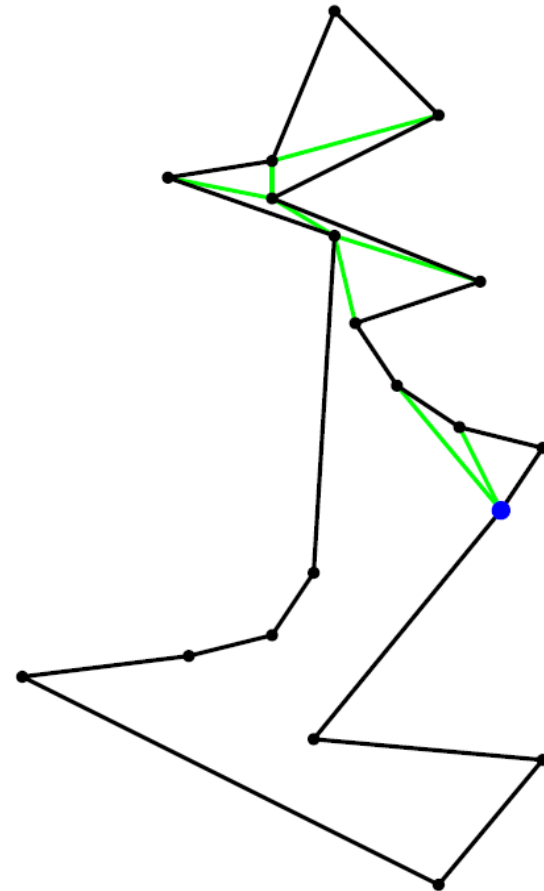
Triangulating a Monotone Polygon

- How to triangulate a y-monotone polygon?



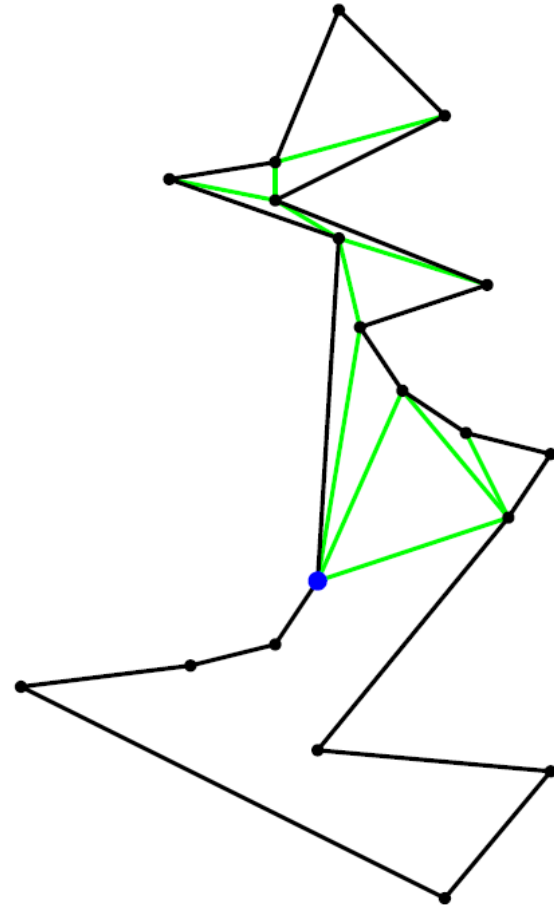
Triangulating a Monotone Polygon

- How to triangulate a y-monotone polygon?



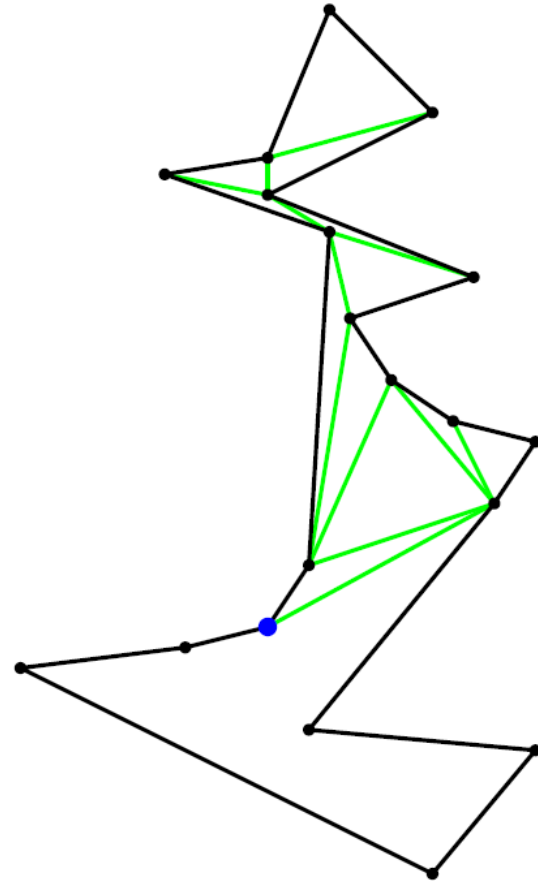
Triangulating a Monotone Polygon

- How to triangulate a y-monotone polygon?



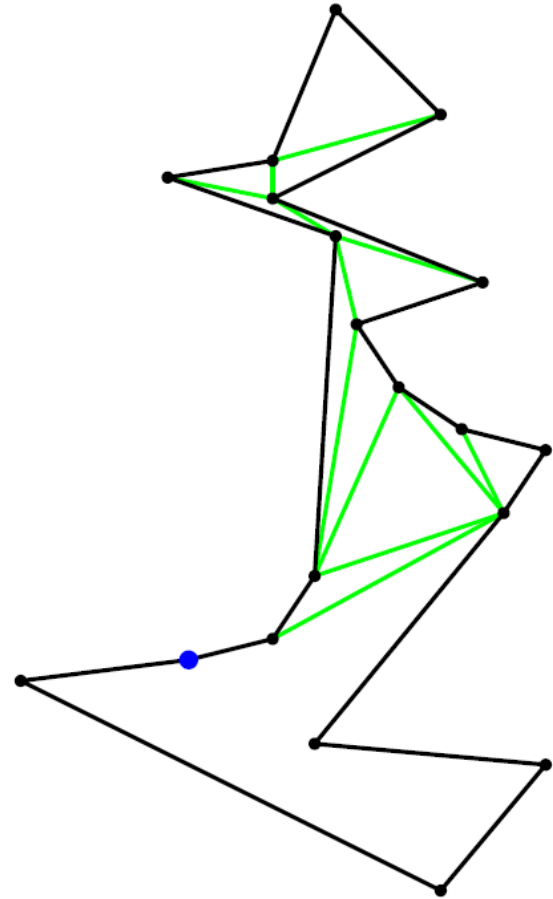
Triangulating a Monotone Polygon

- How to triangulate a y-monotone polygon?



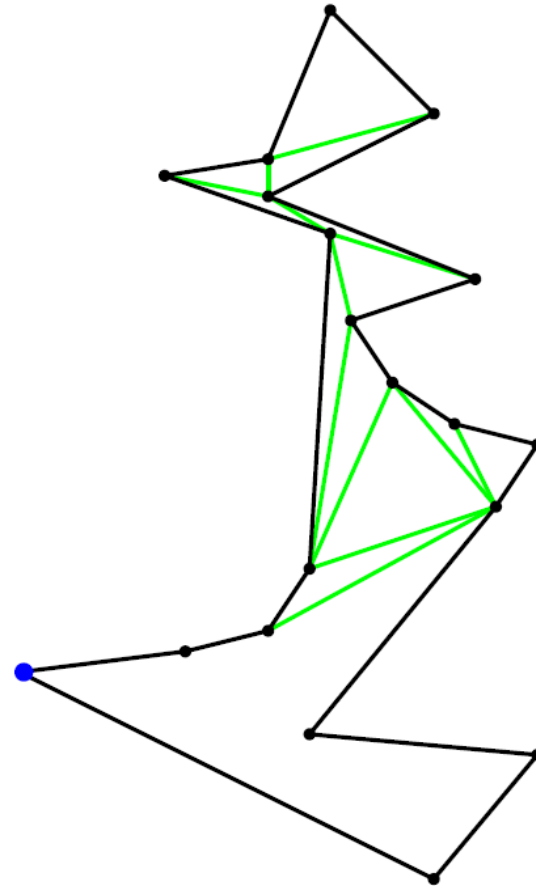
Triangulating a Monotone Polygon

- How to triangulate a y-monotone polygon?



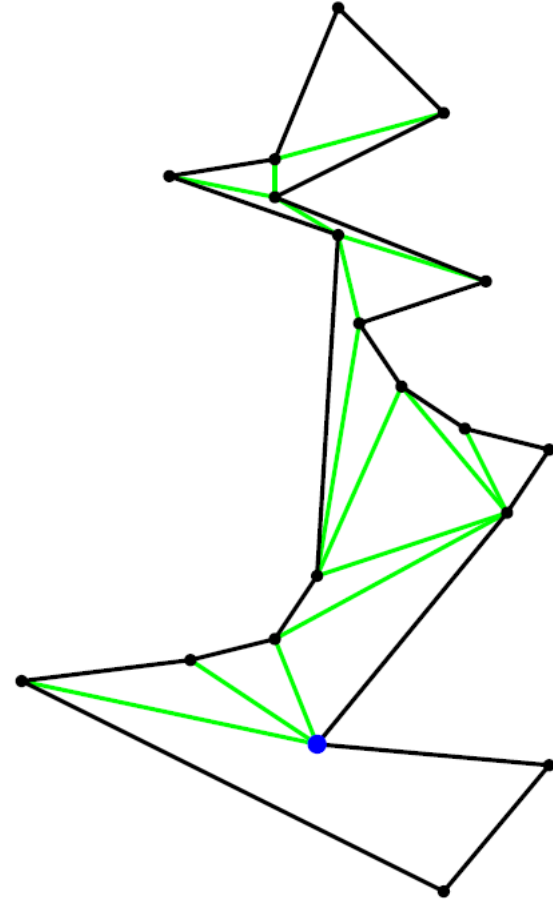
Triangulating a Monotone Polygon

- How to triangulate a y-monotone polygon?



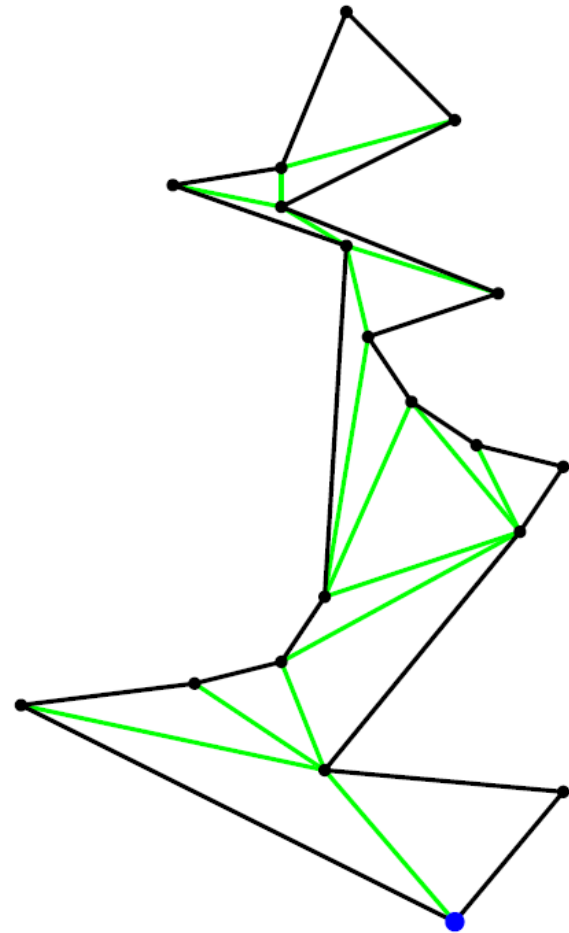
Triangulating a Monotone Polygon

- How to triangulate a y-monotone polygon?



Triangulating a Monotone Polygon

- How to triangulate a y-monotone polygon?



The Algorithm

- Sort the vertices top-to-bottom by a merge of the two chains
- Initialize a stack. Push the first two vertices
- Take the next vertex v , and triangulate as much as possible, top-down, while popping the stack
- Push v onto the stack

Result

- Theorem: A simple polygon with n vertices can be partitioned into y -monotone pieces in $O(n \log n)$ time
- Theorem: A monotone polygon with n vertices can be triangulated $O(n)$ time
- Can we immediately conclude:
- A simple polygon with n vertices can be triangulated in $O(n \log n)$ time ???

Result

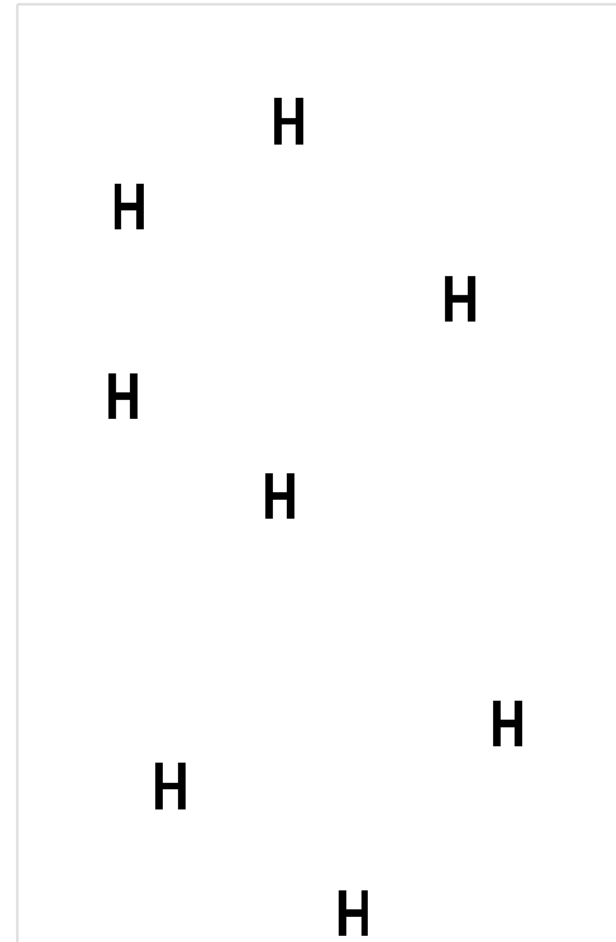
- We need to argue that all y-monotone polygons that we will triangulate have $O(n)$ vertices together
- Initially we had n edges. We add at most $n-3$ diagonals in the sweeps. These diagonals are used on both sides as edges. So all monotone polygons together have at most $3n-6$ edges, and therefore at most $3n-6$ vertices
- Hence we can conclude that triangulating all monotone polygons together takes only $O(n)$ time
- Theorem: A simple polygon with n vertices can be triangulated $O(n \log n)$ time

Voronoi Diagrams

Slides by M. van Kreveld
(Utrecht University)

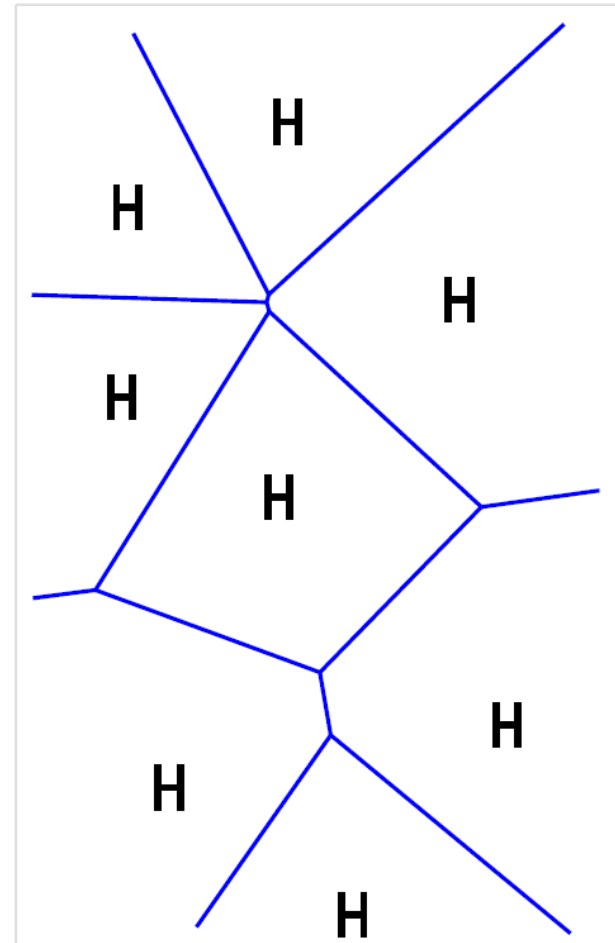
Voronoi Diagram

- Given ambulance posts, where should the ambulance come from in case of an emergency somewhere?



Voronoi Diagram

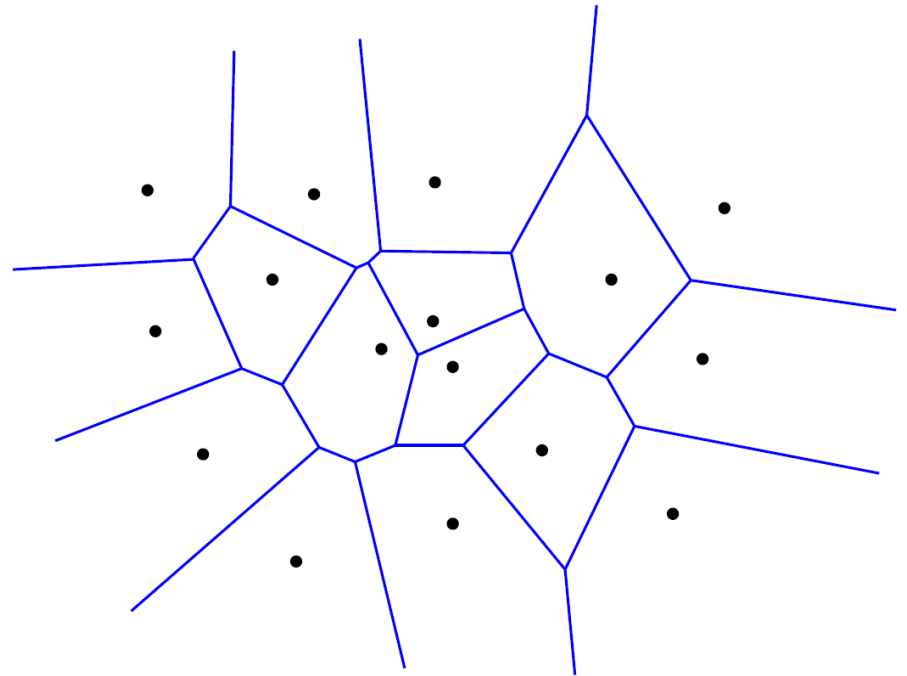
- Given ambulance posts, where should the ambulance come from in case of an emergency somewhere?



Voronoi Diagram

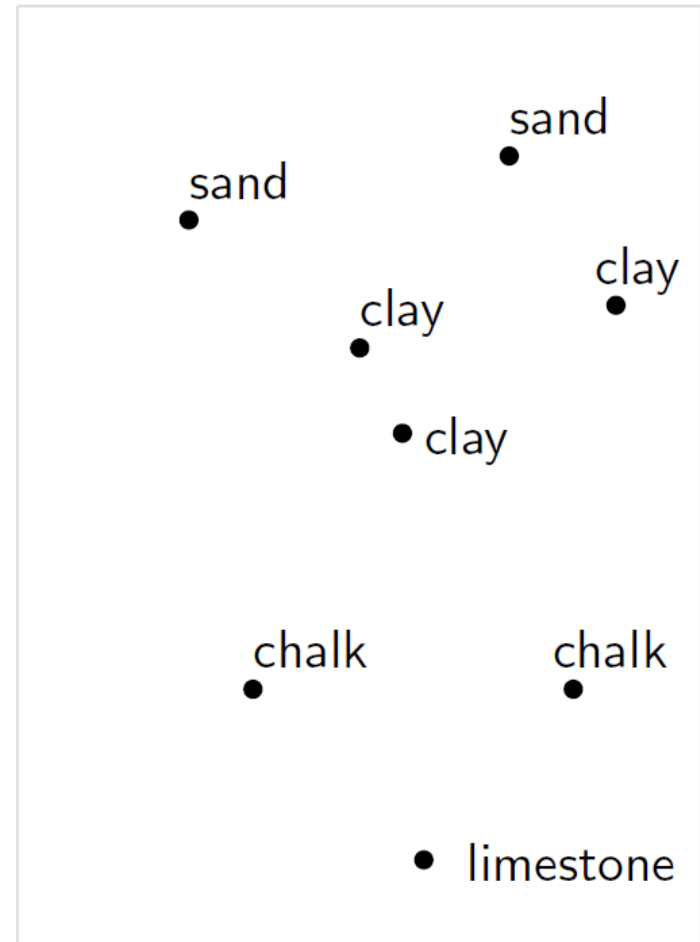
Definition

- The Voronoi diagram induced by a set of points (called sites):
- Subdivision of the plane where the faces correspond to the regions where one site is closest



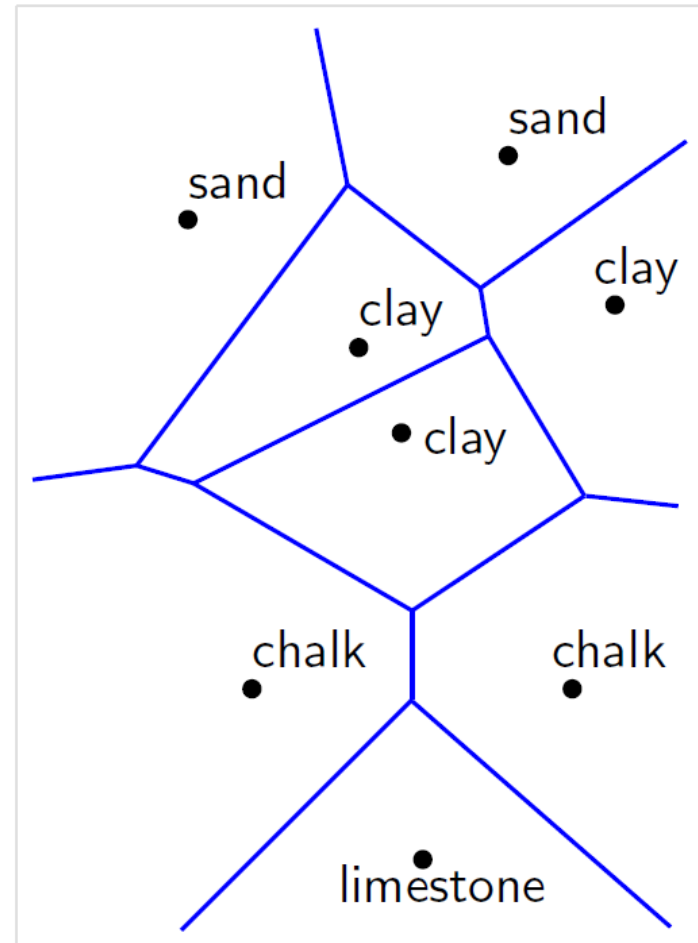
Spatial Interpolation

- Suppose we tested the soil at a number of sample points and classified the results



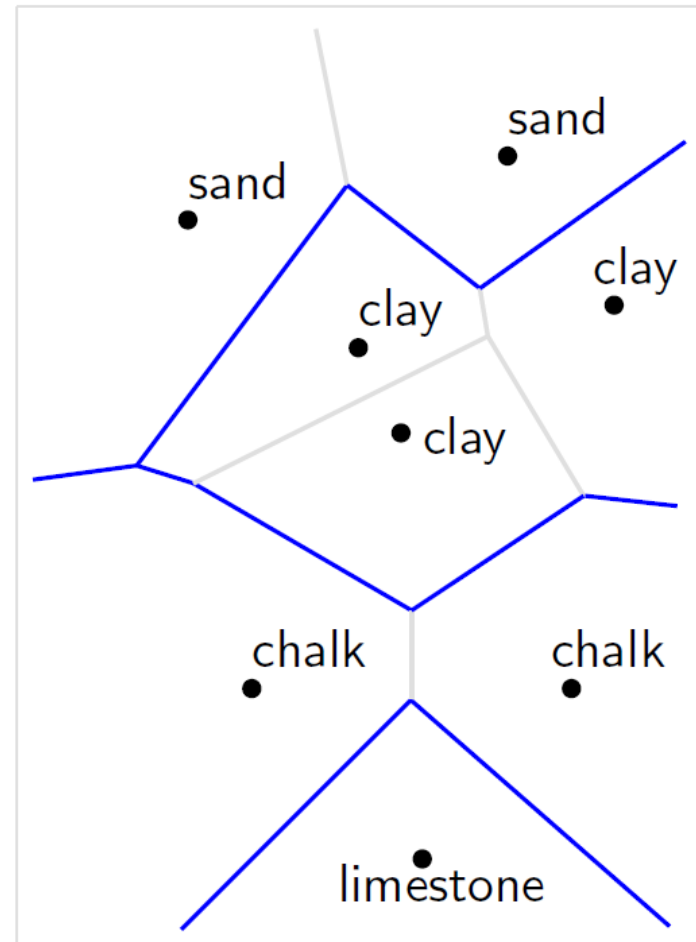
Spatial Interpolation

- Suppose we tested the soil at a number of sample points and classified the results



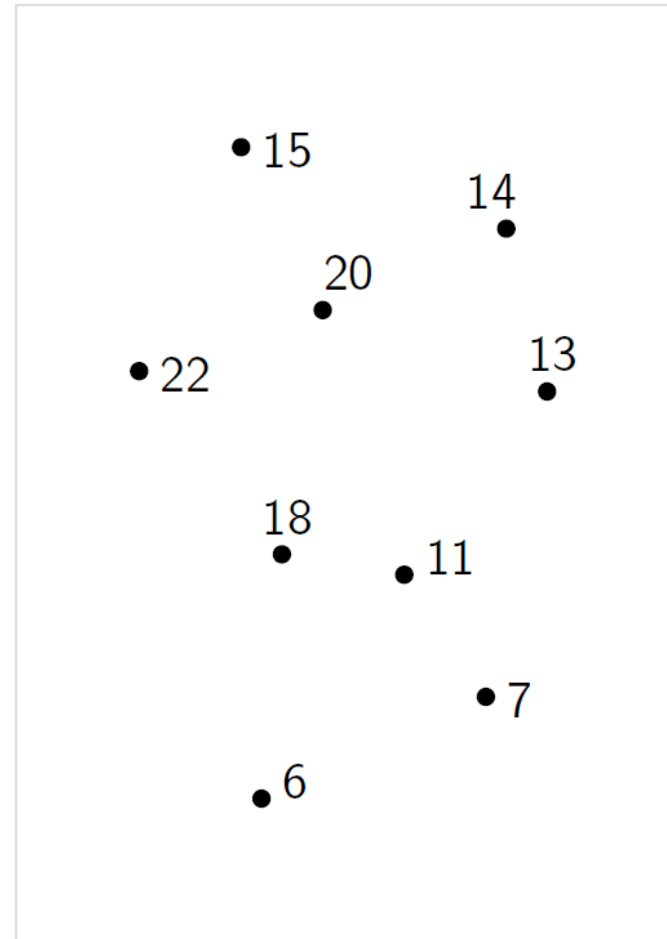
Spatial Interpolation

- Suppose we tested the soil at a number of sample points and classified the results



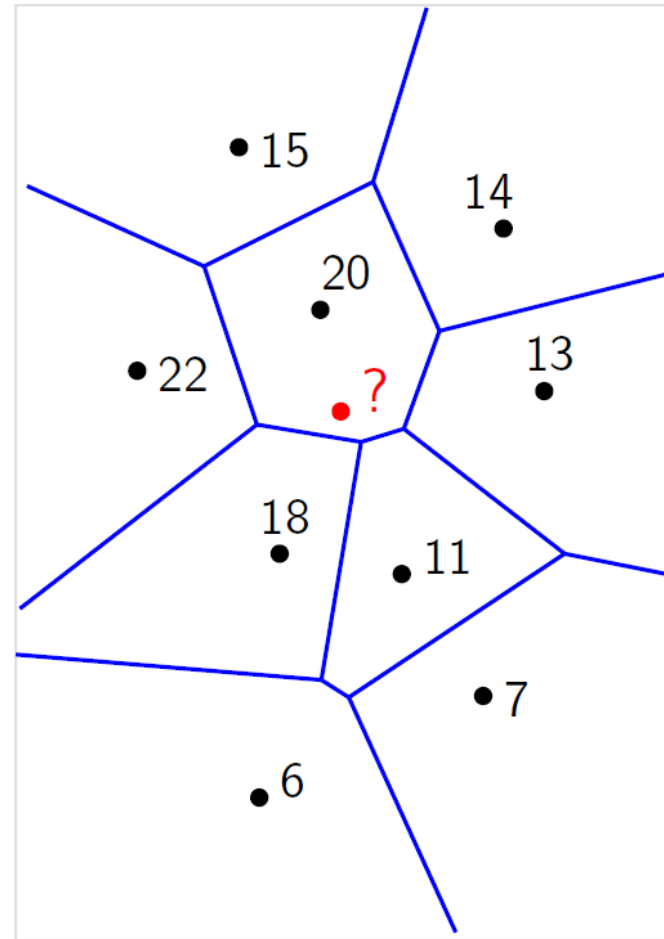
Spatial Interpolation

- Suppose we measured lead concentration at a number of sample points



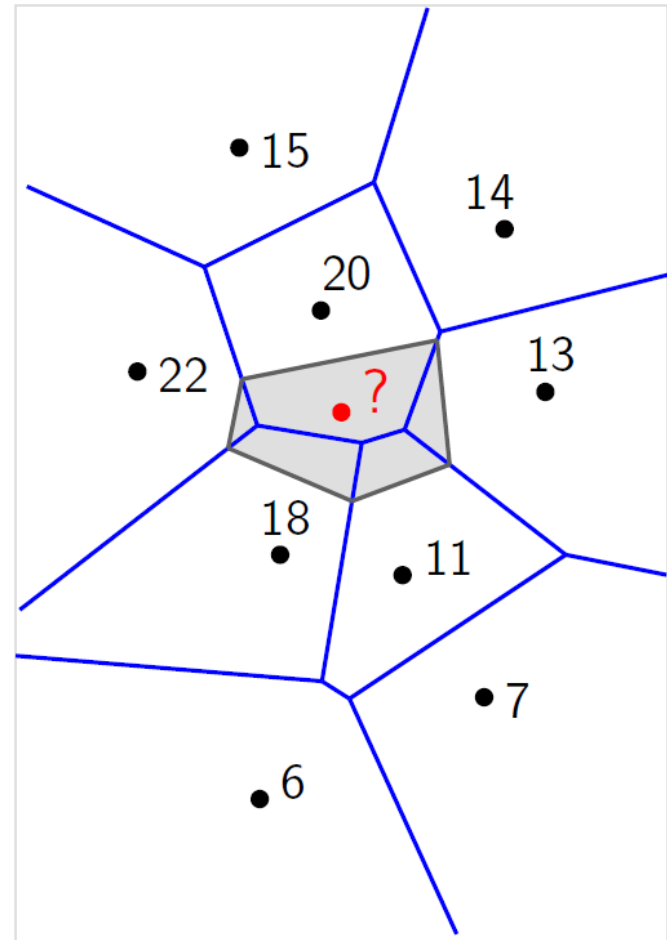
Spatial Interpolation

- Suppose we measured lead concentration at a number of sample points



Spatial Interpolation

- Suppose we measured lead concentration at a number of sample points (natural neighbor interpolation)

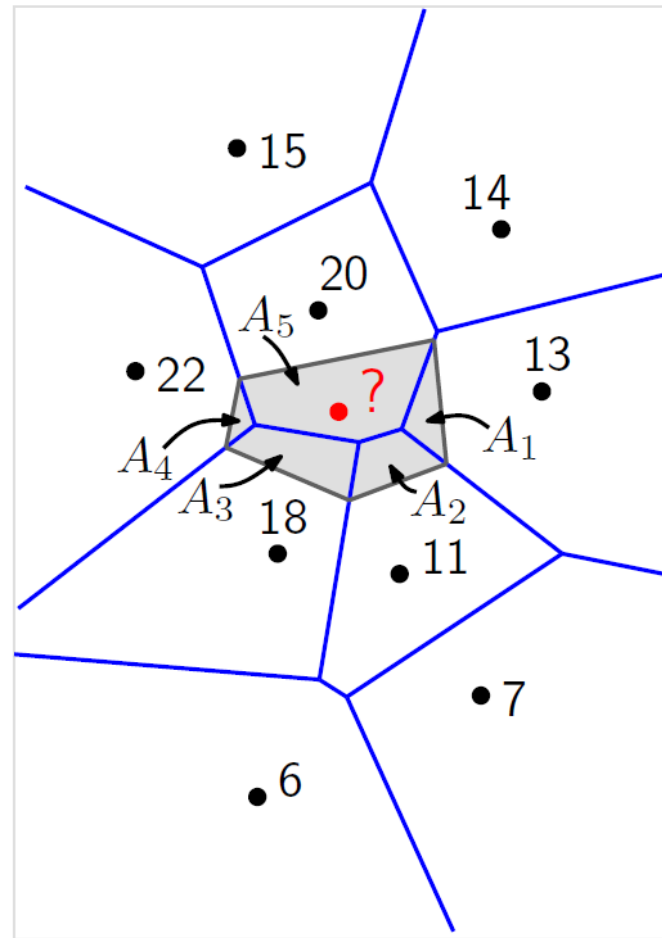


Spatial Interpolation

Natural neighbor interpolation

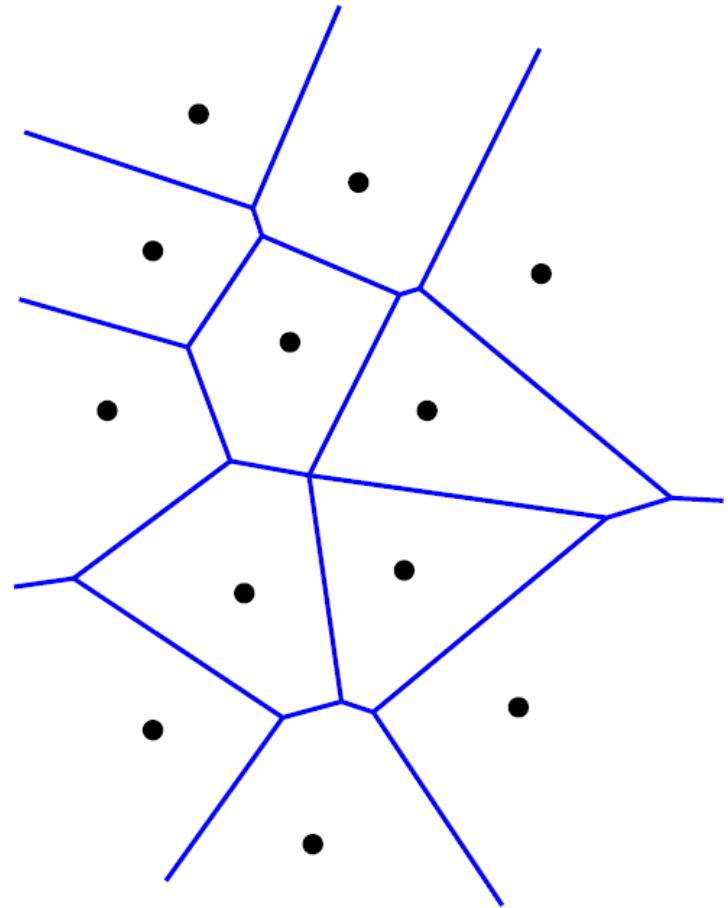
- Let $A_T = A_1 + A_2 + \dots + A_5$
- The interpolated value is:

$$\frac{A_1}{A_T} 13 + \frac{A_2}{A_T} 11 + \dots + \frac{A_5}{A_T} 20$$



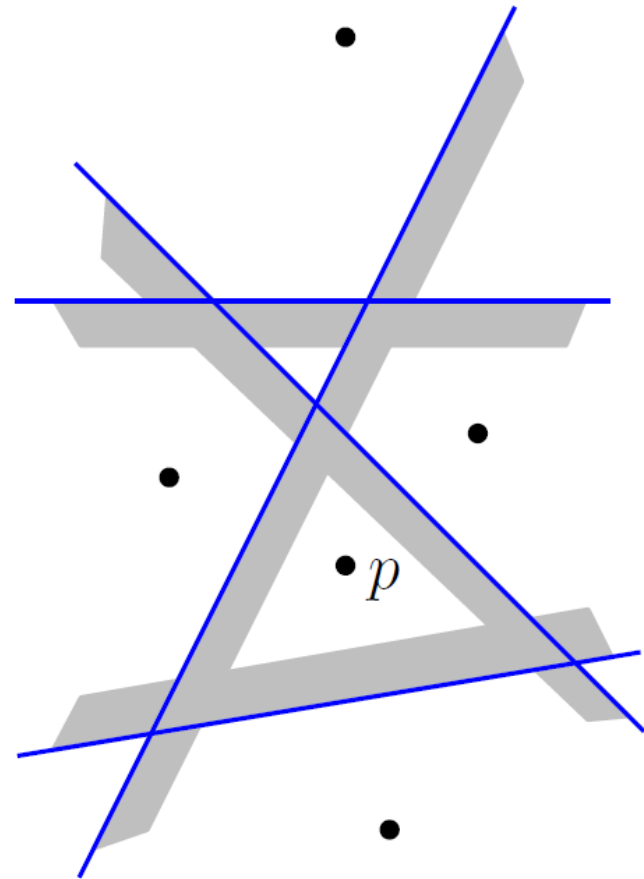
Observations

- Edges are parts of bisectors
- Some edges are half-infinite
- Some cells are unbounded



Every Voronoi cell is the intersection of $n-1$ half-planes, if there are n sites

$\Rightarrow$ all cells are convex and have up to $n-1$ edges in the boundary



Structure

- The Voronoi diagram of n sites has the following structure:
- If all n sites lie on a line, then the Voronoi cell boundaries are parallel lines, so the “graph” is disconnected
- Otherwise, the Voronoi cell boundaries form a connected “graph”

Complexity

- **Theorem:** The Voronoi diagram on n sites in the plane has at most $2n-5$ Voronoi vertices and at most $3n-6$ Voronoi edges (including lines and half-lines)
- **Proof:** If the sites are colinear, then it is trivial
- Otherwise, we will use Euler's formula for planar graphs

Complexity

- Euler's formula for planar graphs: a connected planar graph with n_v vertices, n_e edges, and n_f faces satisfies:

$$n_v - n_e + n_f = 2$$

- However, a Voronoi diagram is not a proper graph

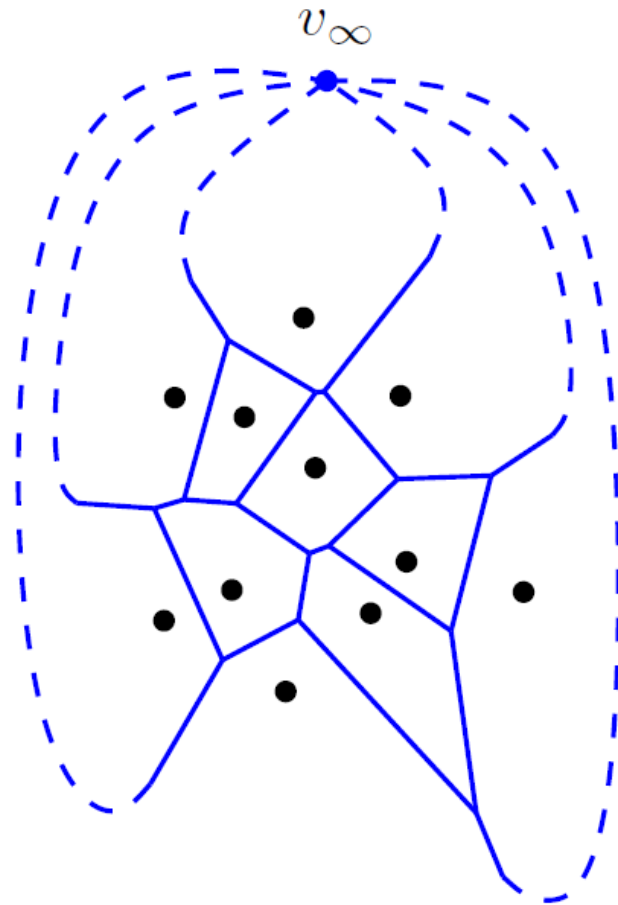
Complexity

We make it proper by connecting all half-infinite edges to a new vertex v_∞

$n_v = \text{no. of Voronoi vertices } VV + 1$

$n_e = \text{no. of Voronoi edges } VE$

$n_f = \text{no. of Voronoi cells} = n$, the number of sites



Complexity

- Substitution in Euler's formula $n_v - n_e + n_f = 2$ gives:

$$(VV + 1) - VE + n = 2$$

- Every edge is incident to exactly 2 vertices, and every vertex is incident to at least 3 edges
- Sum-of-degree-of-all-vertices = 2 VE
- Sum-of-degree-of-all-vertices $\geq 3 VV$
- Thus $2 VE \geq 3 VV$

Complexity

The combination of

$$(V+1) - E + n = 2$$

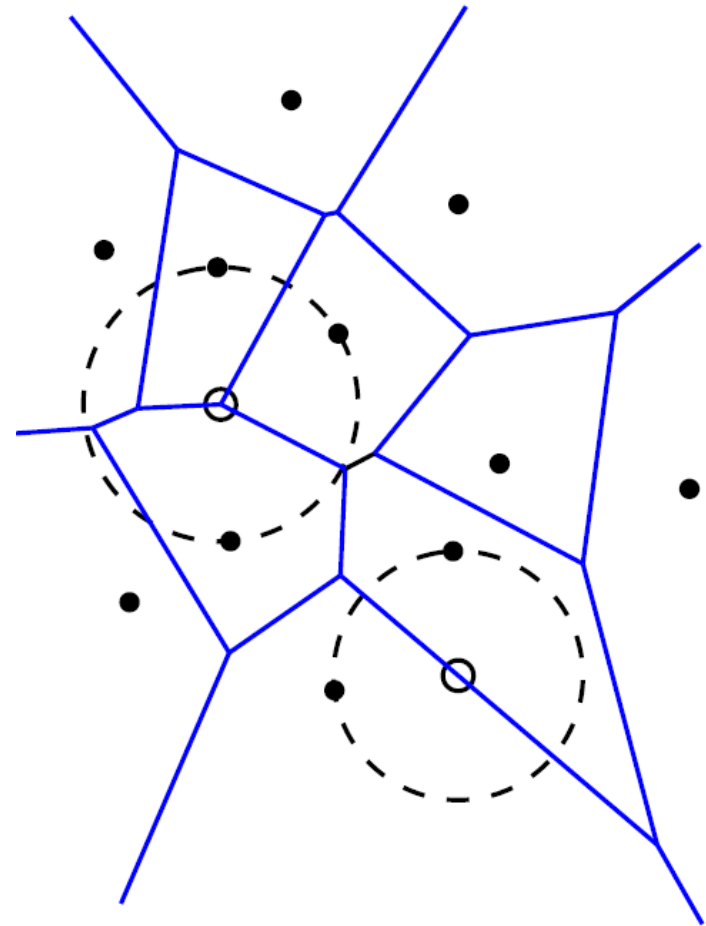
and

$$2E \geq 3(V+1)$$

gives the desired bounds $V \leq 2n-5$ and $E \leq 3n-6$

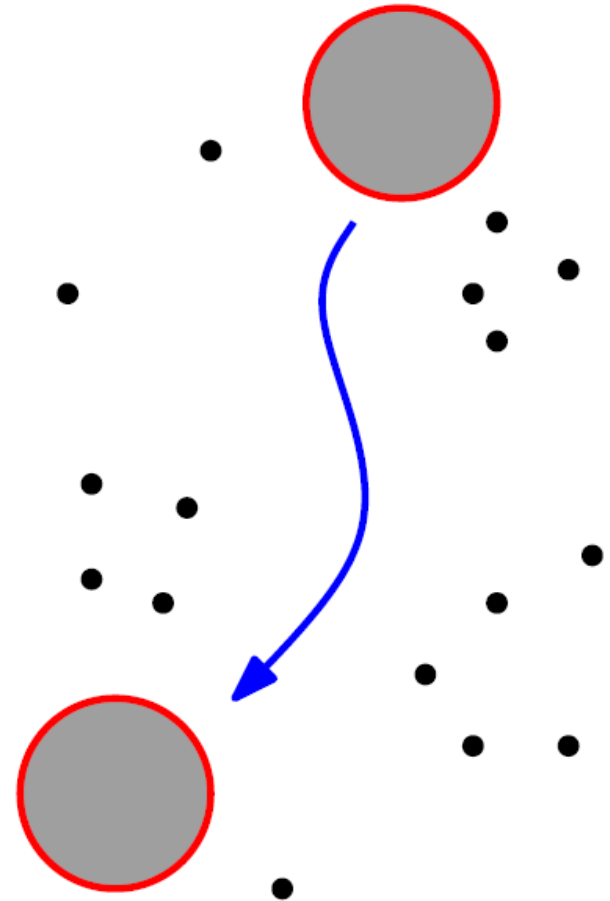
Empty Circle Property

- Every Voronoi vertex is the center of an empty circle through 3 sites
- Every point on a Voronoi edge is the center of an empty circle through 2 sites



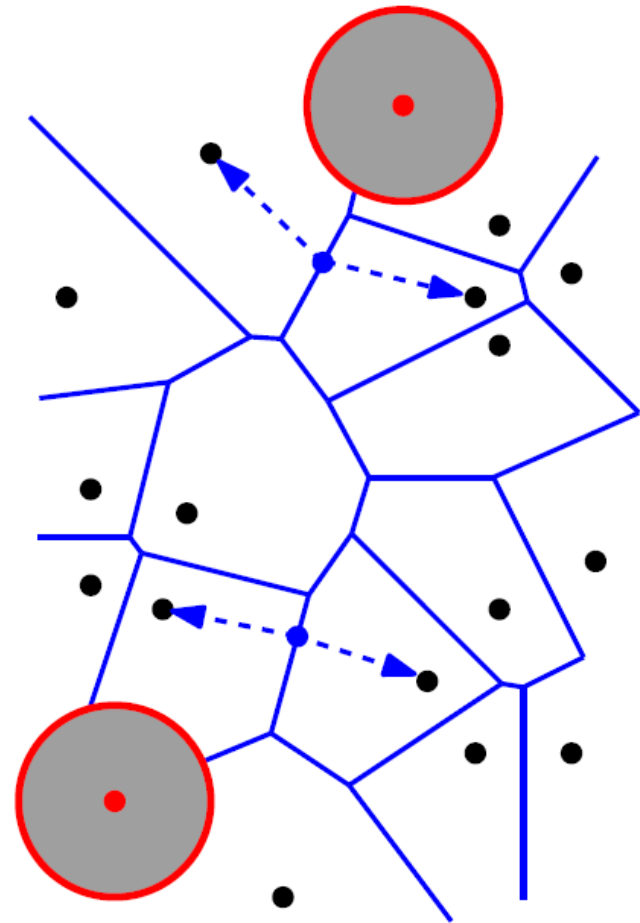
Motion Planning for a Disc

- Can we move a disc from one location to another amidst obstacles?



Motion Planning for a Disc

- Since the Voronoi diagram of point sites is locally “furthest away” from those sites, we can move the disc if and only if we can do so on the Voronoi diagram

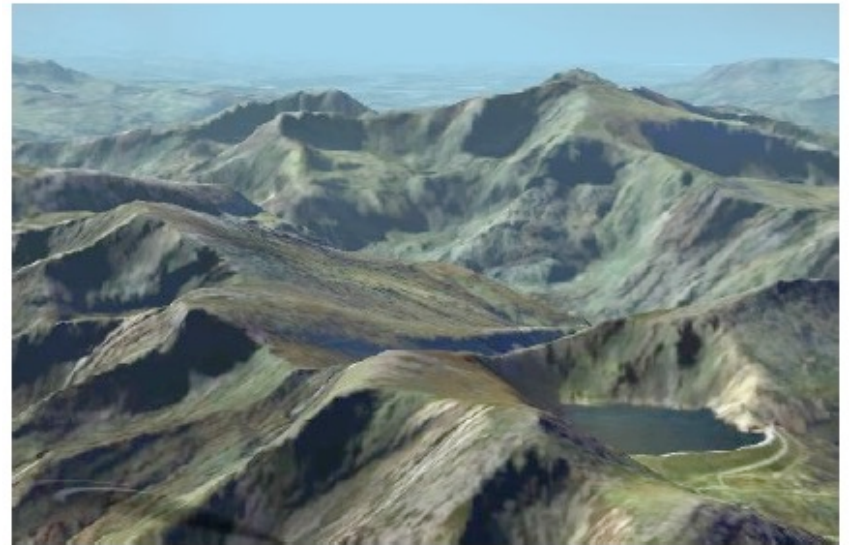


2D Delaunay Triangulations

Slides by M. van Kreveld
(Utrecht University)

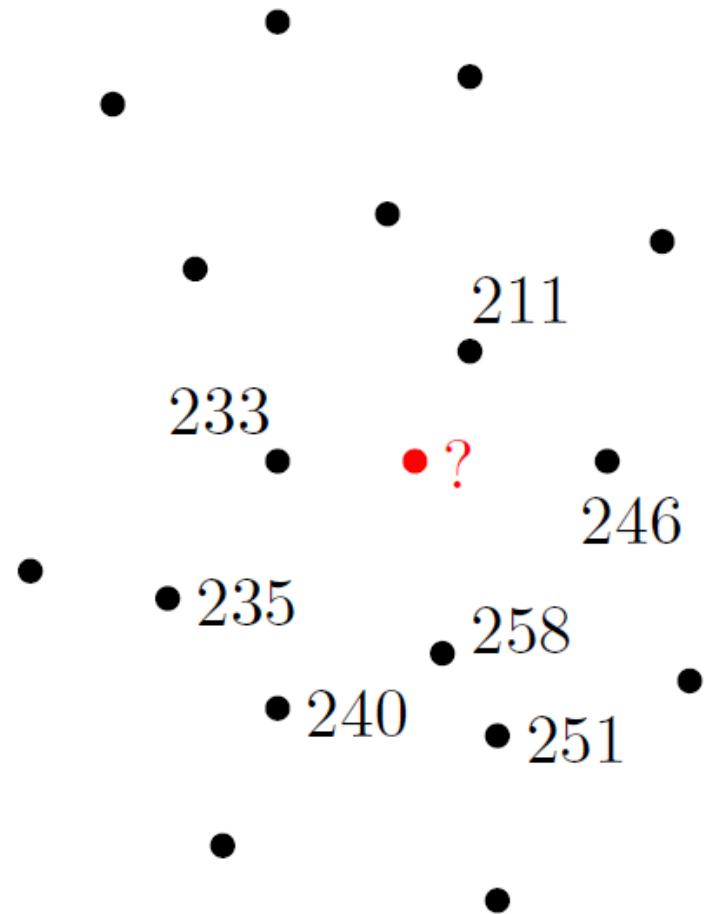
Motivation: Terrains by Interpolation

To build a model of the terrain surface, we can start with a number of sample points where we know the height.



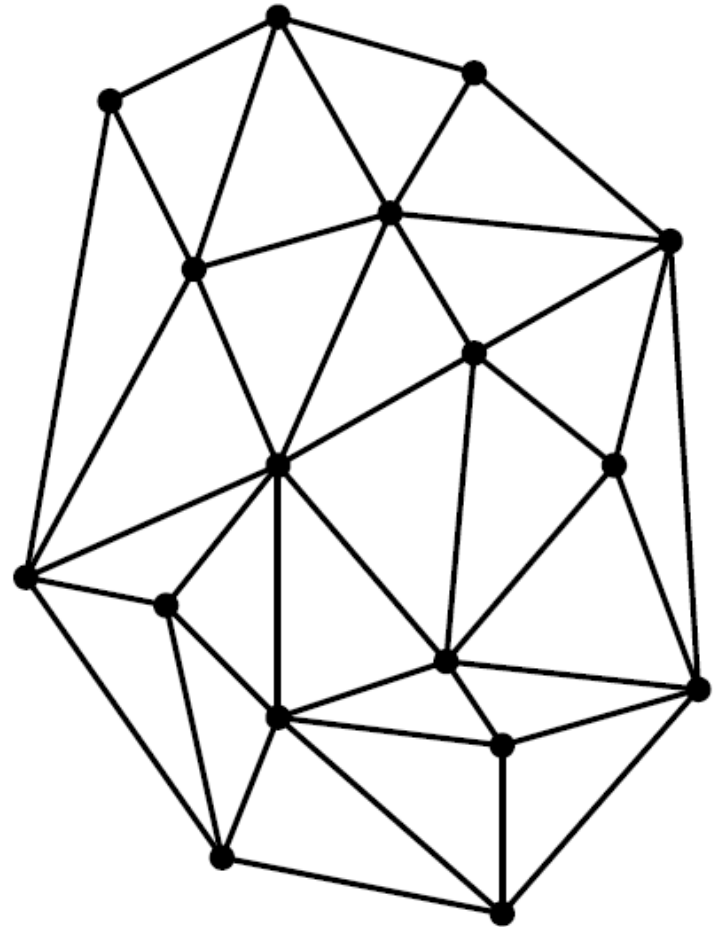
Motivation: Terrains

- How do we interpolate the height at other points?
 - Nearest neighbor interpolation
 - Piecewise linear interpolation by a triangulation
 - Natural neighbor interpolation

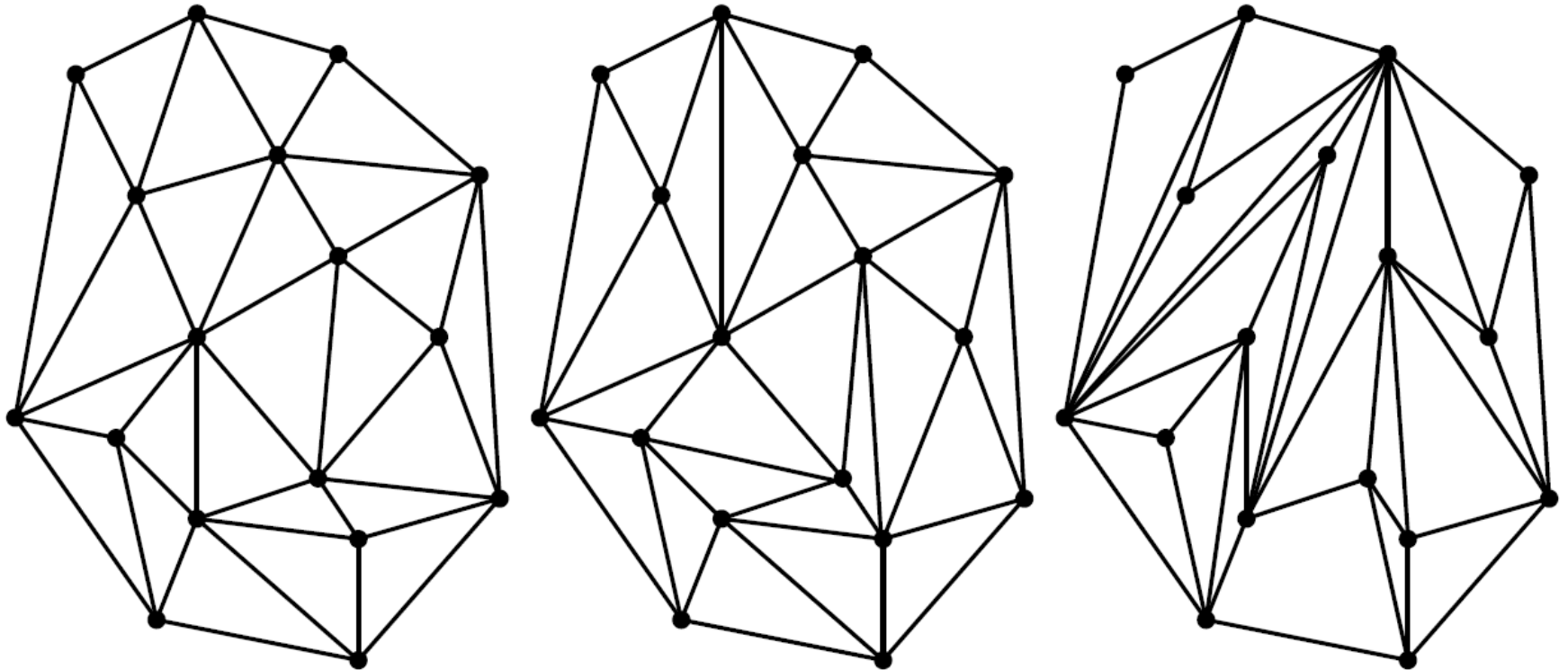


Triangulation

- Let $P = \{p_1, \dots, p_n\}$ be a point set
- A triangulation of P is a maximal planar subdivision with vertex set P
- Complexity:
 - $2n-2-k$ triangles
 - $3n-3-k$ edges
- where k is the number of points in P on the convex hull of P



Which Triangulation?

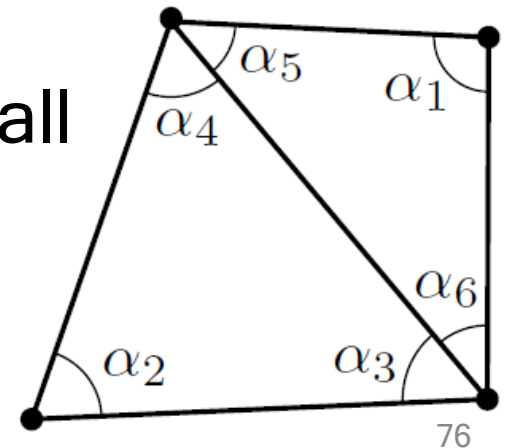


Triangulation

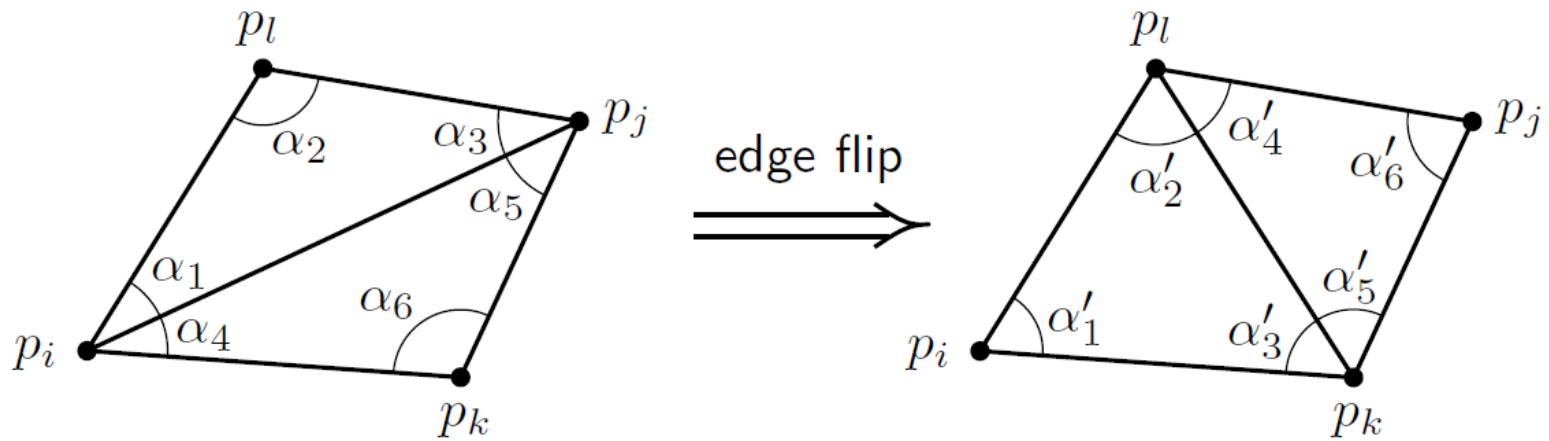
- For interpolation, it is good if triangles are not long and skinny. We will try to use large angles in our triangulation.

Angle Vector of a Triangulation

- Let T be a triangulation of P with m triangles. Its angle vector is $A(T) = (a_1, \dots, a_{3m})$ where $a_1, \dots, a_{3m}$ are the angles of T sorted by increasing value.
- Let T' be another triangulation of P . We define $A(T) > A(T')$ if $A(T)$ is lexicographically larger than $A(T')$
- T is angle optimal if $A(T) \geq A(T')$ for all triangulations T' of P .



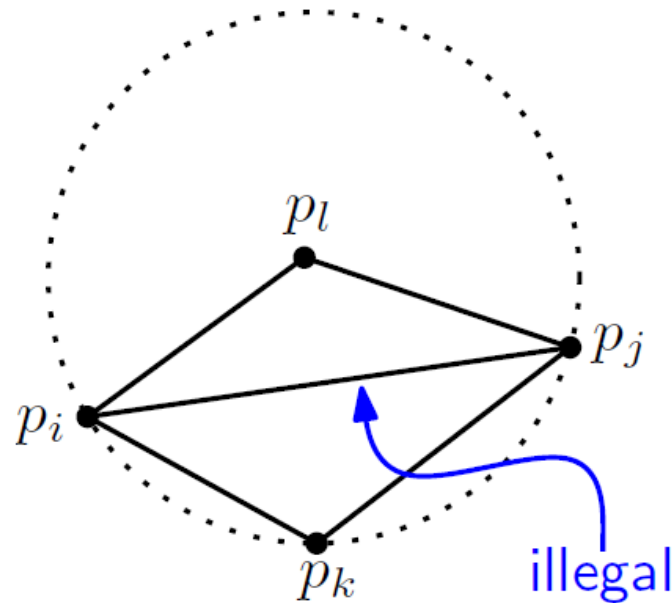
Edge Flipping



- An edge is illegal if $\min\{a_i\} < \min\{a'_i\}$
- Flipping an illegal edge increases the angle vector

Illegal Edges

- An edge $p_i p_j$ is illegal if and only if p_l lies in the interior of the circle C

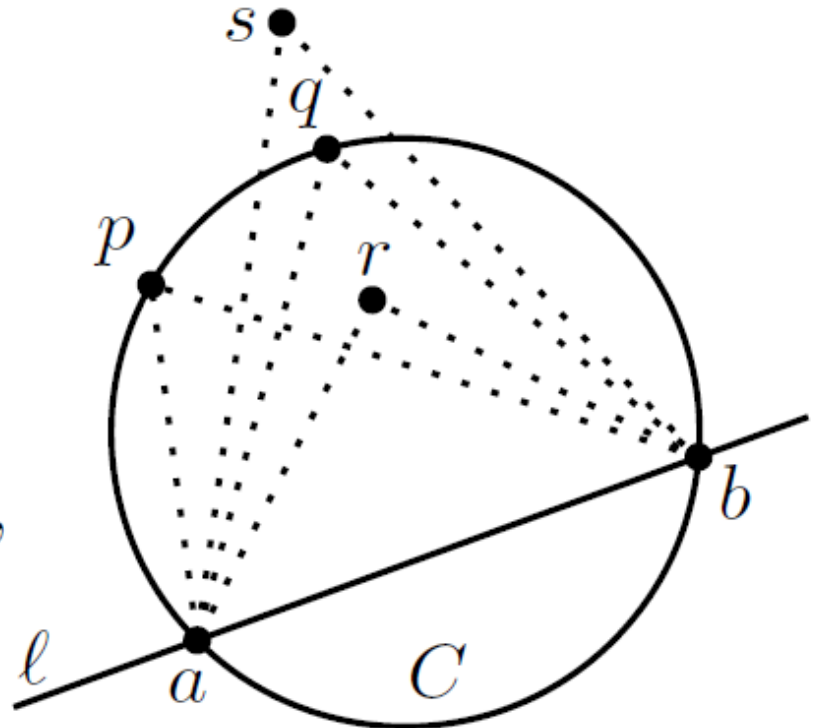


Thales Theorem

- Theorem: Let C be a circle, ℓ a line intersecting C in points a and b , and p, q, r, s points lying on the same side of ℓ . Suppose that p, q lie on C , r lies inside C , and s lies outside C . Then:

$$\angle arb > \angle apb = \angle aqb > \angle asb,$$

- Where $\angle arb$ denotes the smaller angle defined by three points a, b, c .



Legal Triangulations

A **legal triangulation** is a triangulation that does not contain any illegal edge.

Algorithm LEGALTRIANGULATION($\mathcal{T}$)

Input. A triangulation $\mathcal{T}$ of a point set P .

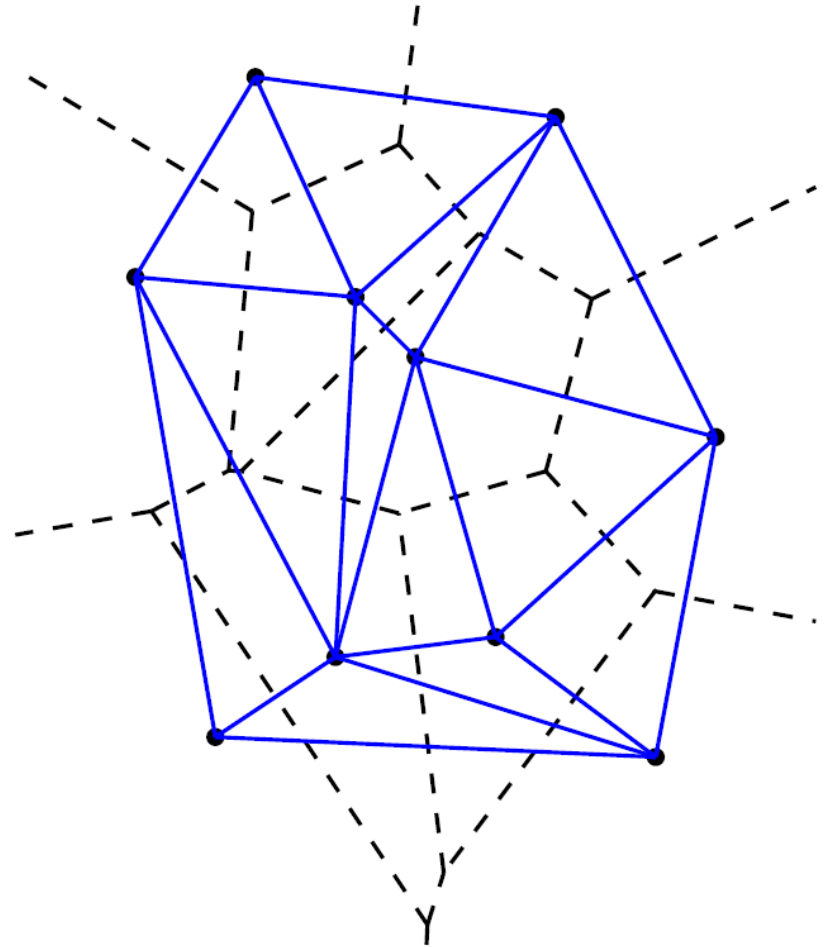
Output. A legal triangulation of P .

1. **while** $\mathcal{T}$ contains an illegal edge $\overline{p_i p_j}$
2. **do** (* Flip $\overline{p_i p_j}$ *)
3. Let $p_i p_j p_k$ and $p_i p_j p_l$ be the two triangles adjacent to $\overline{p_i p_j}$.
4. Remove $\overline{p_i p_j}$ from $\mathcal{T}$, and add $\overline{p_k p_l}$ instead.
5. **return** $\mathcal{T}$

Question: Why does this algorithm terminate?

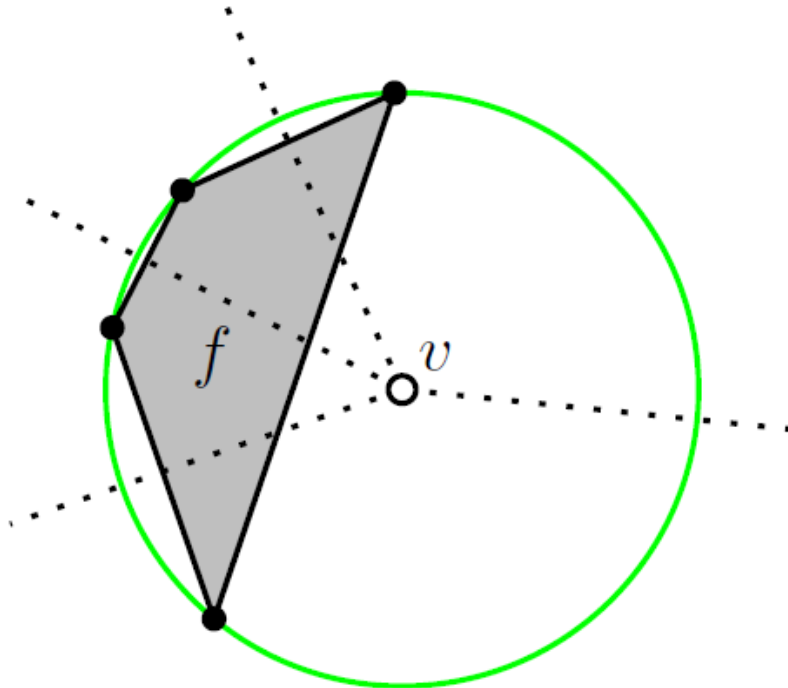
Voronoi Diagram and Delaunay Graph

- Let P be a set of n points in the plane
- The Voronoi diagram $\text{Vor}(P)$ is the subdivision of the plane into Voronoi cells $V(p)$
- Let G be the dual graph of $\text{Vor}(P)$
- The Delaunay graph $\text{DG}(P)$ is the straight line embedding of G



Delaunay Triangulation

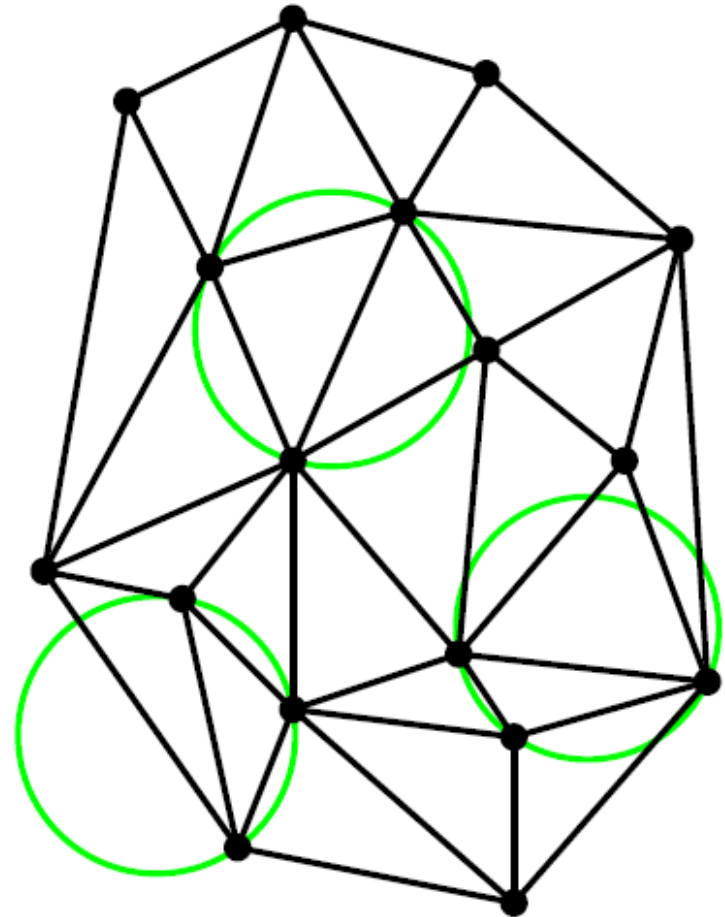
- If the point set P is in general position, then the Delaunay graph is a triangulation



When discussing [Delaunay triangulations](#) in the plane, a set of [points](#) in the [plane](#) is said to be in general position only if no four of them lie on the same circle.

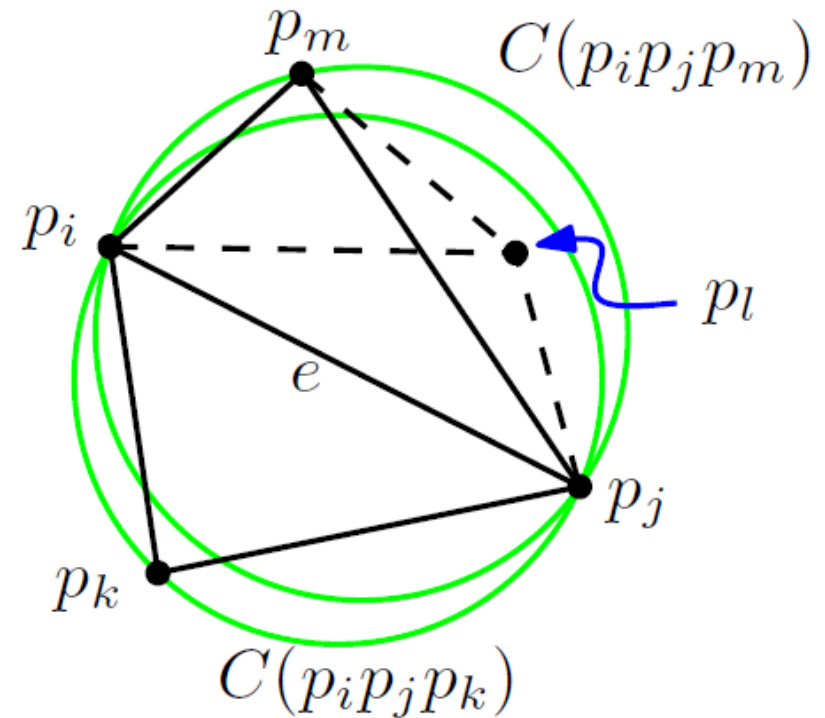
Empty Circle Property

Theorem: Let P be a set of points in the plane, and let T be a triangulation of P . Then T is a Delaunay triangulation of P if and only if the circumcircle of any triangle of T does not contain a point of P in its interior.



Delaunay Triangulations and Legal Triangulations

Theorem: Let P be a set of points in the plane. A triangulation T of P is legal if and only if T is a Delaunay triangulation.



Computing Delaunay Triangulations

- There are several ways to compute the Delaunay triangulation:
 - By iterative flipping from any triangulation
 - By plane sweep
 - By randomized incremental construction
 - By conversion from the Voronoi diagram
- The last three run in $O(n \log n)$ time [expected] for n points in the plane

In Circle Test

- Assume that no four sites are co-circular
- In circle test is equivalent to a determinant computation
 - Assume that abcd define a counterclockwise convex polygon (abc is the original triangle)
 - If not, d lies inside triangle and the test fails
 - d lies in circumcircle if and only if the following determinant is positive (if it is 0, the points are co-circular)

$$\text{inCircle}(a, b, c, d) = \det \begin{pmatrix} a_x & a_y & a_x^2 + a_y^2 & 1 \\ b_x & b_y & b_x^2 + b_y^2 & 1 \\ c_x & c_y & c_x^2 + c_y^2 & 1 \\ d_x & d_y & d_x^2 + d_y^2 & 1 \end{pmatrix} > 0.$$

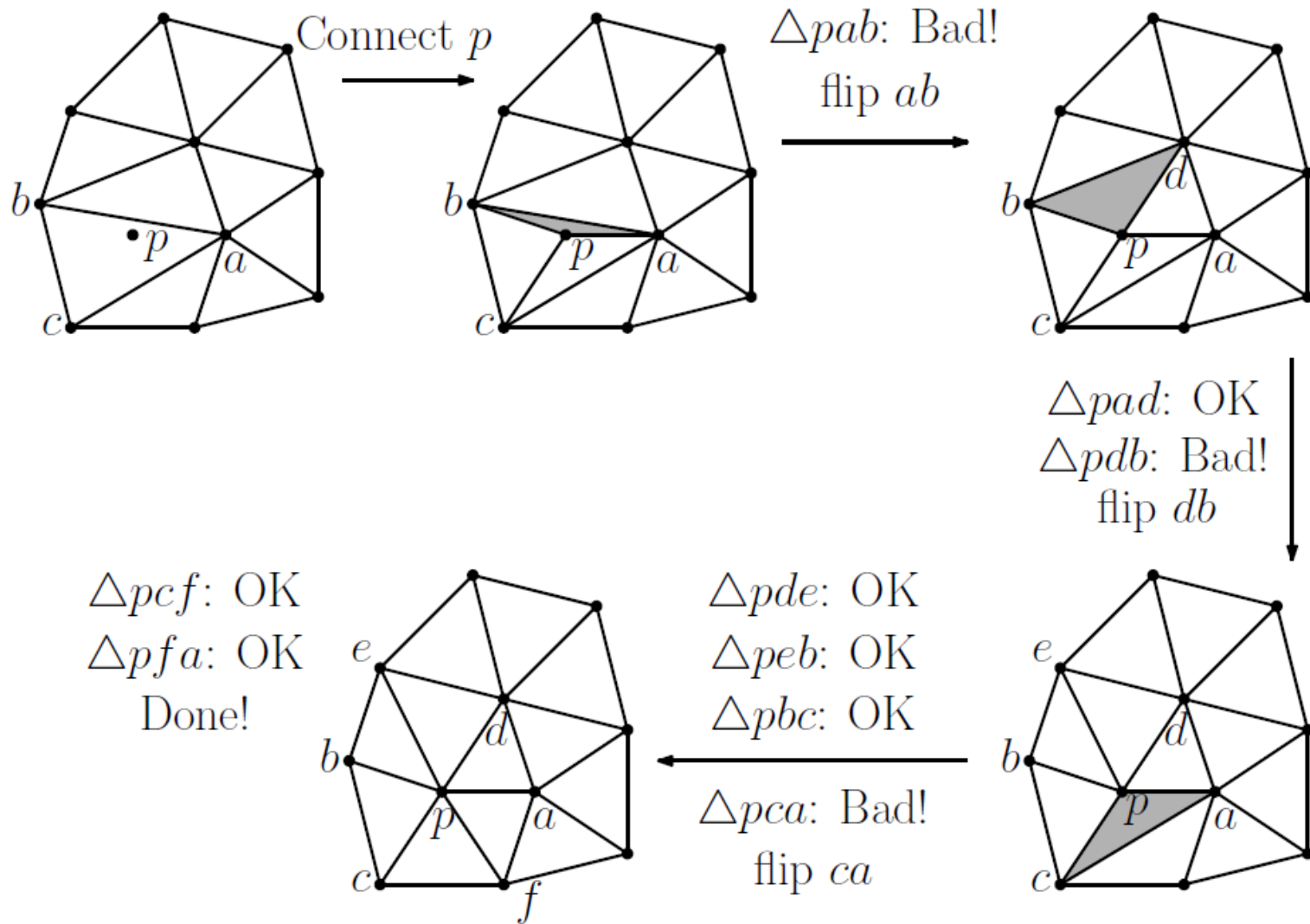
Incremental Update

- Create a non-Delaunay triangulation and fix it
 - Join new point with the vertices of the triangle that contains it
 - Flip edges as needed
 - Both can be done in $O(1)$
-
- Initialize by enclosing all points in a very large triangle (its vertices must lie outside of all circumcircles of final triangulation)

Incremental Update

- For each new point p , we have created three new triangles
- For each of the triangles that have been added, we check the vertex of the triangle that lies on the opposite side of the edge that does not include p
- If this vertex fails the in circle test, then we swap the edge creating two new triangles that are adjacent to p
- We repeat the same test with these triangles

Incremental Update Example



Details

- This is only a sketch of algorithm
- We would need to prove that a triangulation that is locally Delaunay is also globally Delaunay
- Each time triangles are deleted and new triangles are made, uninserted points must be re-bucketed in $O(1)$ time per point and each point is expected to be re-bucketed $O(\log n)$ times

Locally-flattened Delaunay

Pros:

- Good approximation to Delaunay

- Easily modifiable to be incremental / dynamic

- Not so sensible to noise

Pros:

- Can have problems with more complicated surfaces

Gabriel 2–Simplex criterion

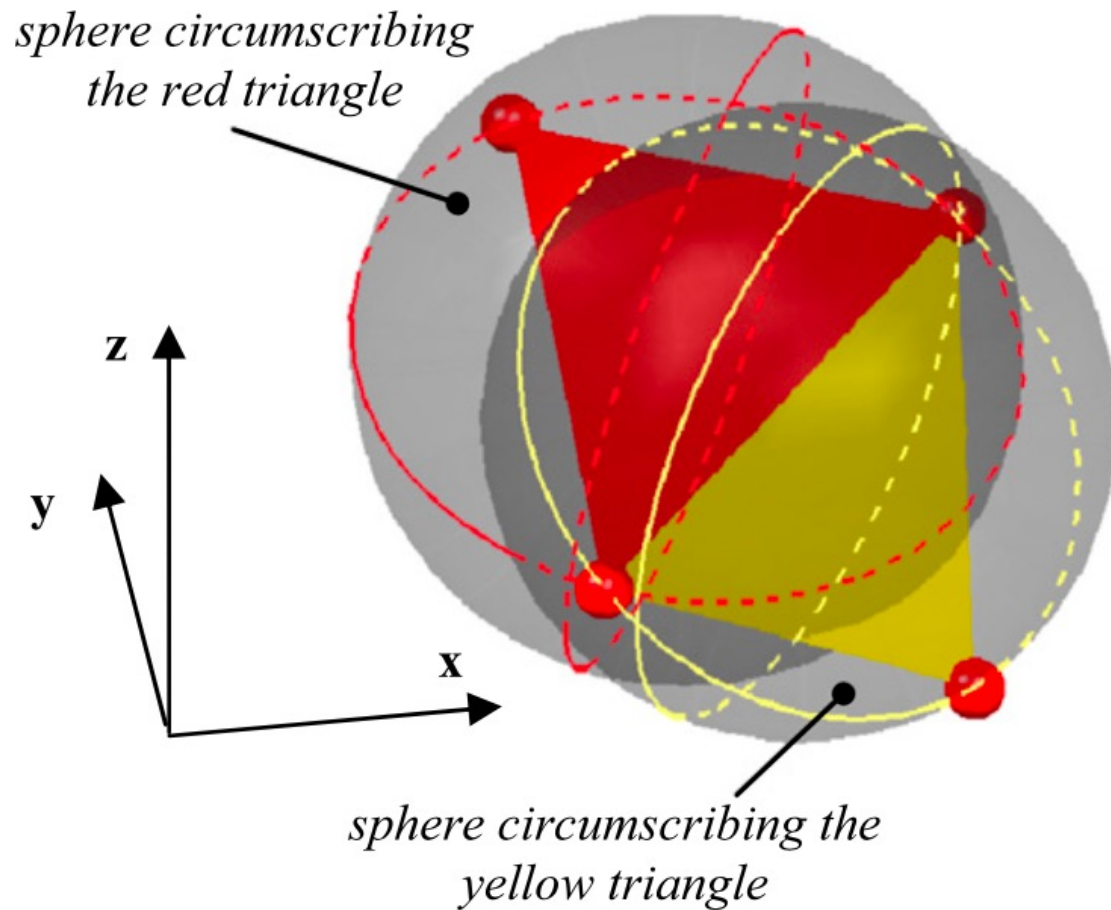


Fig. 1. Examples of G2S triangles; the two spheres respectively circumscribing the vertices of the red and the yellow triangles are empty.

Incremental Mesh Reconstruction

- Pick a seed triangle from the mesh
- Run G2S Triangulation

Picking a Seed Triangle

Step 1: Pick a random 3D point of the cloud

Step 2: Build edge with NN

Step 3: Define sphere around edge.

Center is edge midpoint

Radius proportional to edge length

Step 4: if Not G2S Triangle Goto Step 1

Step 5: if Not On_Surface Goto Step 1

Step 6: Report Seed Triangle

Triangle Surface Test

- Step 1: Build infinite cylinder passing through the triangle vertices and having its axis parallel to the normal of the triangle
- Step 2: The points contained in this cylinder must be either all above or all under the triangle

Triangle Surface Test

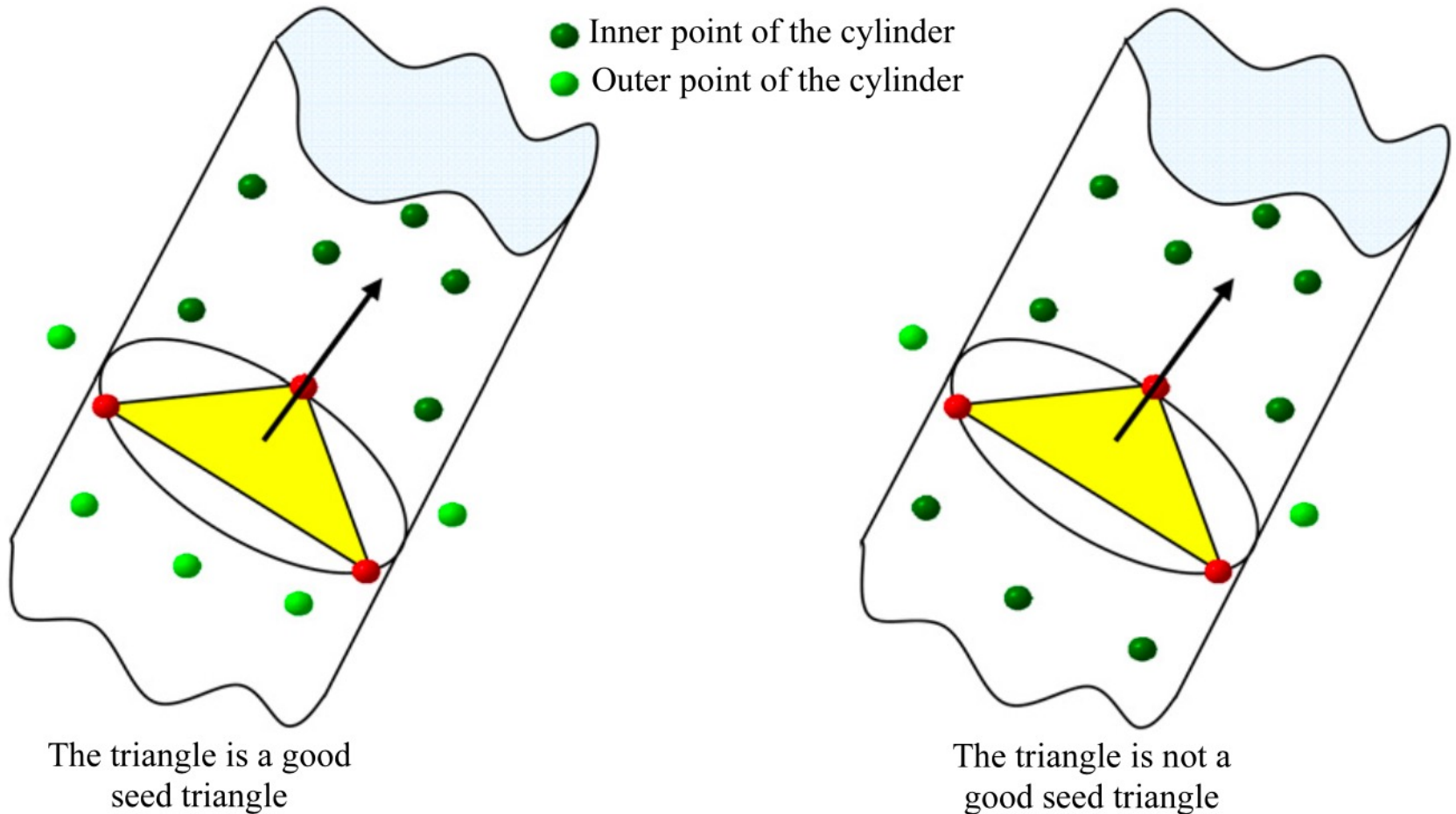


Fig. 3. The cylinder test to select a good seed triangle.

G2S Triangulation

- Push Seed triangle edges
- Pop a free edge e_f (starts from the seed triangle)
- Identify of all the candidate points p near e_f
 - Build a sphere centered on triangle plane and away from the edge, radius is user defined

G2S Triangulation

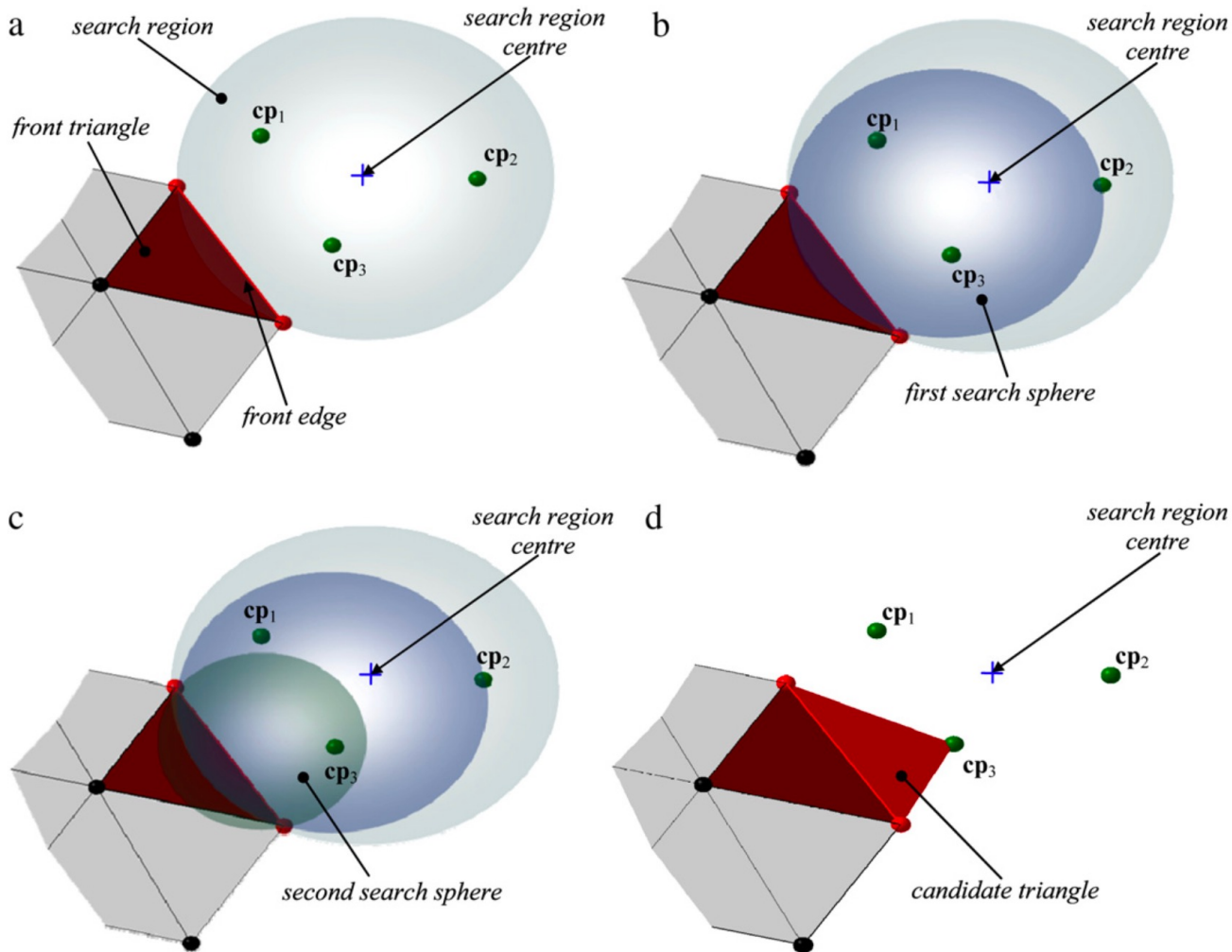


Fig. 5. Explanation of the strategy for the selection of a reference point with a view perpendicular to the front triangle.

G2S Triangulation

- Push Seed triangle edges
- Pop a free edge e_f (starts from the seed triangle)
 - Identify of all the candidate points p near e_f
 - For each point p
 - Build a sphere containing the e_f and p
 - Break and keep point if the sphere is empty
 - Check topology of the new triangle edges
 - If new then push edges